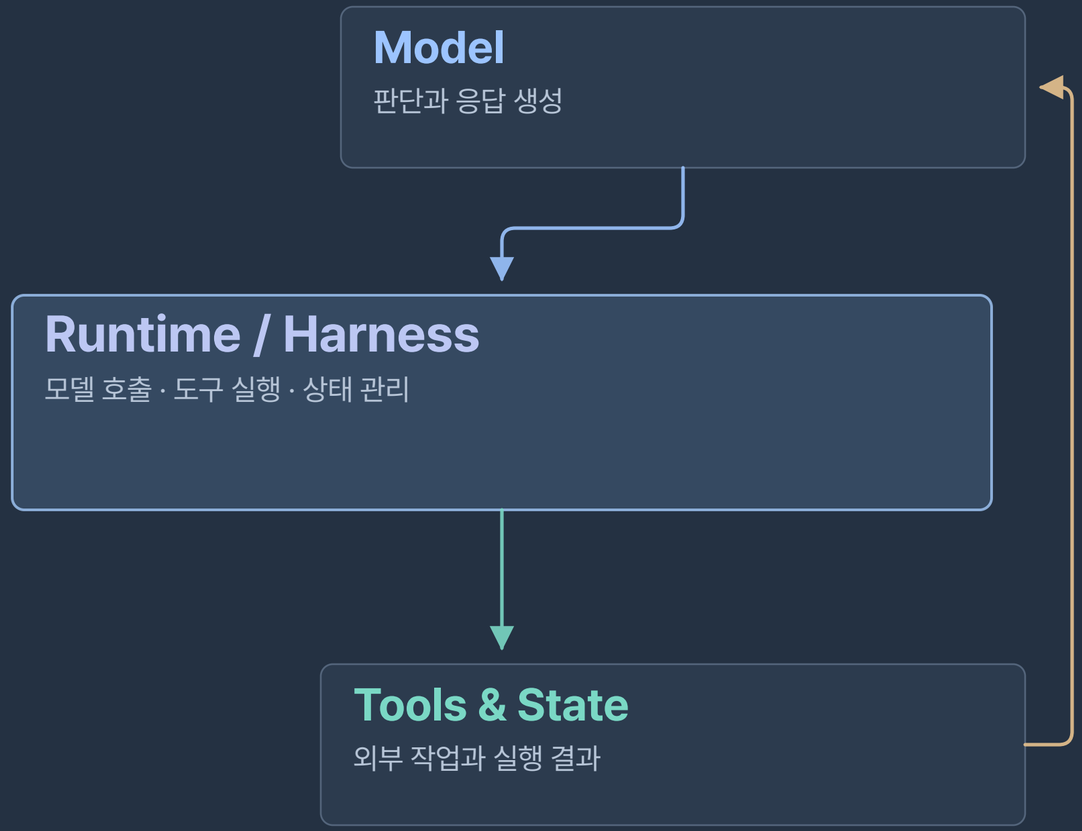
Agent Framework

세부 구조를 살펴봅니다

2차 세미나

Hermes **Claude Code** **PydanticAI**

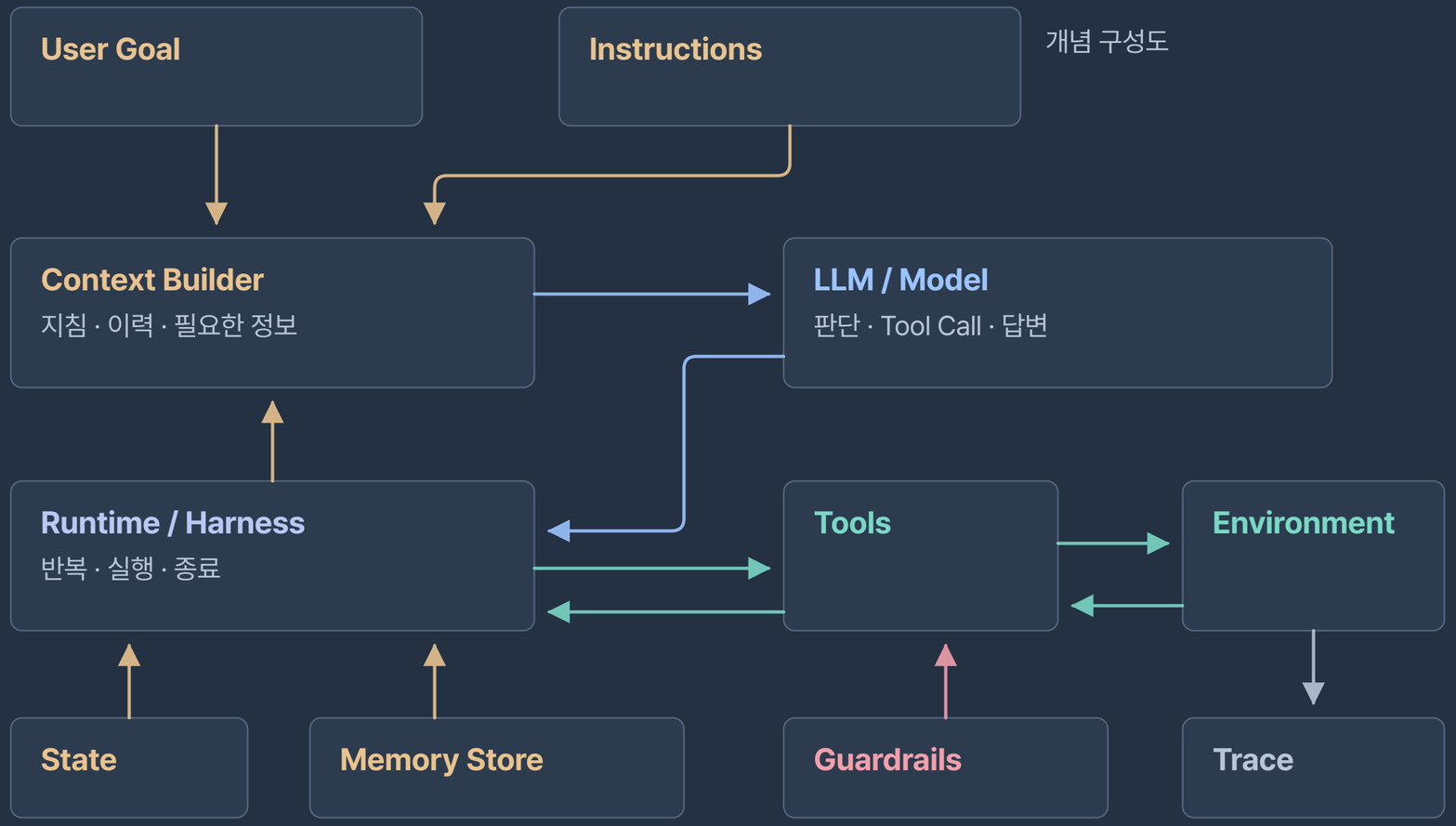


Agent Framework의 구성

목표가 모델 호출과 도구 실행으로 이어지는 과정

Agent를 이루는 주요 구성요소

목표와 지침을 읽고
모델과 도구를 연결해
작업을 이어갑니다.



각 기술이 이 구조를 어떻게 구현하는지 살펴봅니다.

2차 세미나 진행 계획

1차 세미나의 내용을 기반으로 실제 기술별 세부 사항을 살펴봅니다.



모델 개발·서빙
AI Studio · vLLM · SGLang

강준혁

Agent 개발·워크플로
DeepAgents · LangChain · LangGraph

박건

Agent Harness·SDK
Hermes · PydanticAI · OpenAI · Claude

남우성

3차에서는 직접 개발하고, 결과를 검증하며 회고합니다.

Agent Harness와 SDK

완성된 Agent를 확장하거나, 코드로 필요한 기능을 구성합니다.

Agent Harness

갖춰진 실행환경에서 시작

Hermes
범용 Agent Harness

Claude Code
코딩 작업을 위한 Harness

Agent SDK

코드로 Agent 구성

PydanticAI
Pydantic 기반 Agent SDK

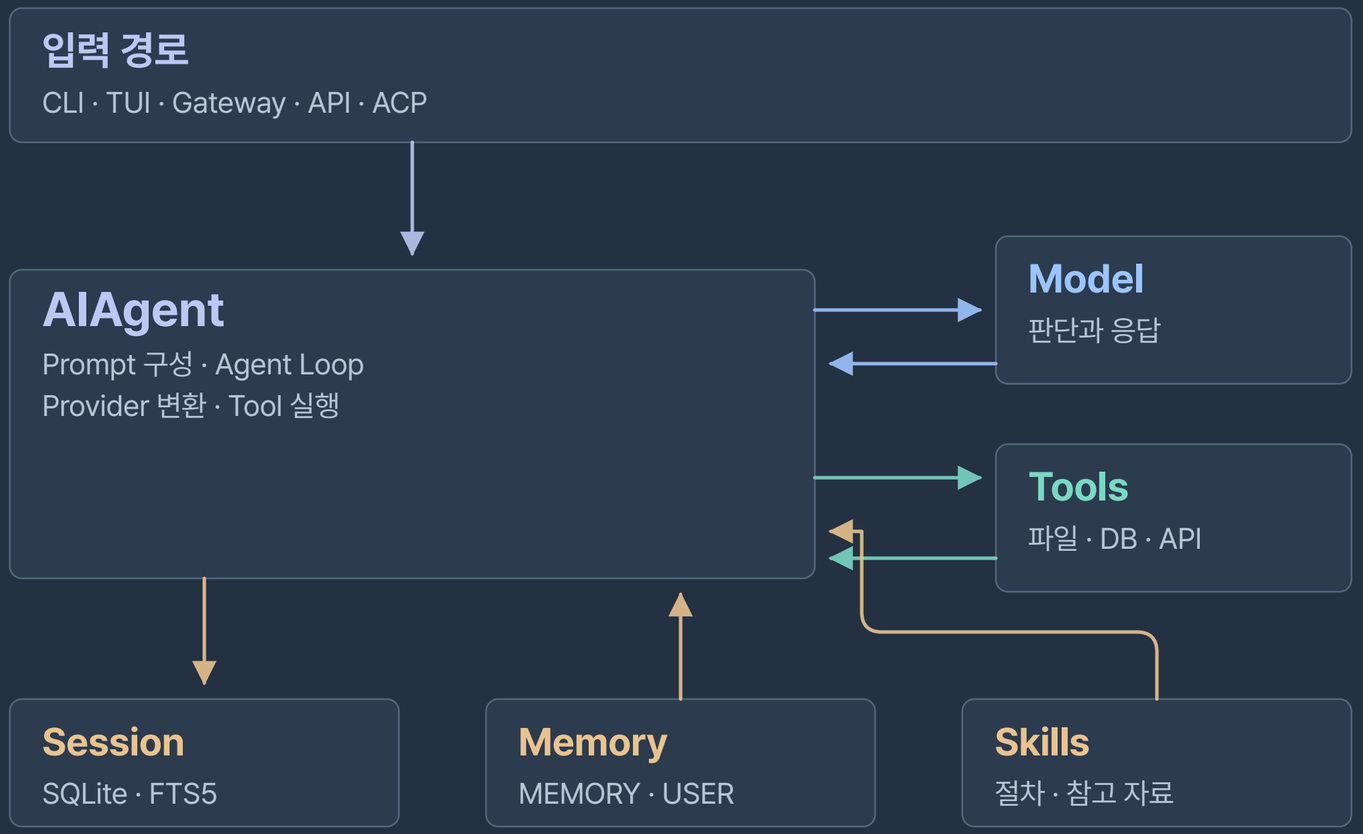
OpenAI Agents SDK

Claude Agent SDK

Google ADK

Microsoft Agent Framework

Hermes와 PydanticAI를 중심으로 살펴봅니다.



Hermes

범용 Agent Harness

모델과 도구를 연결하고
대화와 작업을 이어가는
실행환경

전체 구조를 먼저 보고, 각 기능의 동작을 살펴봅니다.

Hermes 내부 구성과 설명 순서

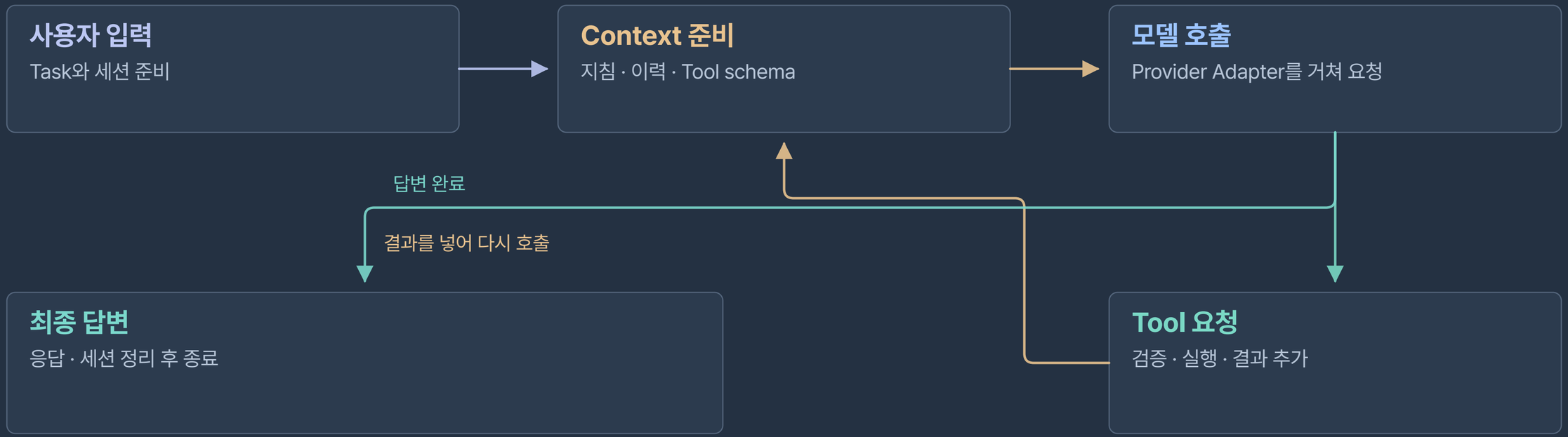
범용 Agent Harness에서 실제 작업이 이어지는 과정을 살펴봅니다.

실행 루프	반복 조건 · 중단 · 예산 · 오류
Context와 기억	System Prompt · Session · Memory · 검색
Tools와 Skills	등록 · 노출 · 검증 · 승인 · 자체 절차 수정
안전장치와 기록	Guardrail · 정책 확장 · Trace 수집·활용
긴 작업과 위임	Prefix Cache · 압축 · 병렬 Tool · Subagent
예약 실행과 활용	Cron · 사용 명령 · 실제 관찰 Demo

실행 흐름을 먼저 보고, 입력·도구·상태·확장 기능을 차례로 확인합니다.

사용자 요청 한 건의 실행 과정

한 Turn 안에서 모델 호출과 Tool 실행이 반복됩니다.



중단 · 예산 소진 · 오류 각 종료 원인에 맞춰 기록과 응답을 정리

사용자 요청 수와 모델 호출 수는 다릅니다.

Hermes는 언제 다시 모델을 부를까?

바깥 루프 한 번을 위에서 아래로 읽습니다.

conversation_loop.py · v0.22.5 · 실제 조건 및 호출 축약

```
while prelude_ok and (
    (calls < max_iterations and remaining > 0)
    or grace_call
):
    _run_phase(begin_iteration, agent, s)
    _run_phase(prepare_iteration, agent, s)
    _run_phase(assemble_api_request, agent, s)
    _run_phase(run_preflight_gate, agent, s)
    _run_phase(announce_api_call, agent, s)
```

① 들어갈 조건 확인
준비 단계가 중단되지 않았고
횟수와 예산이 남거나 정리 호출이 허용됨

② begin_iteration
중단 플래그를 확인한 뒤 호출 번호를
올리고 예산 한 회를 사용

③ prepare_iteration
이번 반복의 메시지와 상태를 준비

④ assemble_api_request
메시지와 도구를 공급자 요청 형식으로 조립

⑤ run_preflight_gate
보내기 직전 검사. 결과에 따라
전송 전 종료하거나 다음 반복으로 이동

⑥ announce_api_call
모델을 호출하기 전에 진행 상황을 알림

먼저 호출할 수 있는지 확인하고, 실제로 보낼 요청을 준비합니다.

모델 응답을 받고 루프를 잇는 코드

앞 장의 전송 준비 직후부터 도구 실행 또는 답변까지 따라갑니다.

conversation_loop.py · v0.22.5 · 실제 함수 호출 축약

```
_run_api_retry_loop(agent, s)
_run_phase(apply_retry_restarts, agent, s)
_run_phase(normalize_model_response, agent, s)
if s.assistant_message.tool_calls:
    _run_phase(run_tool_round, agent, s)
else:
    _run_phase(finish_text_response, agent, s)
```

① _run_api_retry_loop
모델 응답을 얻는 구간. 통신 재시도는
이 호출 내부에서 처리

② apply_retry_restarts
모델 전환 등으로 요청을 다시 준비해야
하면 바깥 루프로 돌아감

③ normalize_model_response
받은 응답을 공통 메시지로 정리해
도구 호출 목록을 읽을 수 있게 함

④ run_tool_round
예: A17 조회 → 반환값을 이력에 추가 →
다음 모델 요청이 그 값을 읽음

⑤ finish_text_response
도구 호출이 없으면 텍스트 종료 경로.
응답 상태에 따른 예외 분기는 Sources 참고

Hermes는 응답을 읽고 도구 결과를 다음 요청에 넣거나, 답변을 정리합니다.

루프가 멈추는 세 가지 이유

어디에서 멈추는지와 실행 뒤에 남는 결과를 나눠 봅니다.

Hermes v0.22.5 · 실제 분기 위치를 묶은 의사코드

```
if agent._interrupt_requested:
    return _verdict("break")
# 호출 중: handle_api_interrupt(...)
```

```
if agent._budget_grace_call:
    agent._budget_grace_call = False
elif not agent.iteration_budget.consume():
    return _verdict("break")
```

```
while s.retry_count < s.max_retries:
    # 실패 원인에 따라 분기
    handle_api_error(...)
```

중단
새 호출 전에는 중단 표시를 확인.
호출 중 InterruptedError는 별도로 처리.

호출 예산 소진
max_turns 기본 null. 설정된 상한을
소진하면 조건부 정리 호출만 허용.

모델 요청 오류
api_max_retries 기본 3.
원인에 따라 재시도·대체 모델·실패 처리.

중단 뒤에도 이미 실행된 도구의 외부 효과는 남을 수 있습니다.

기본 지시

SOUL · 모델/도구 안내 · 고정 Skill

작업 환경

프로젝트 규칙 · workspace · 호출자 지시

기억과 확장

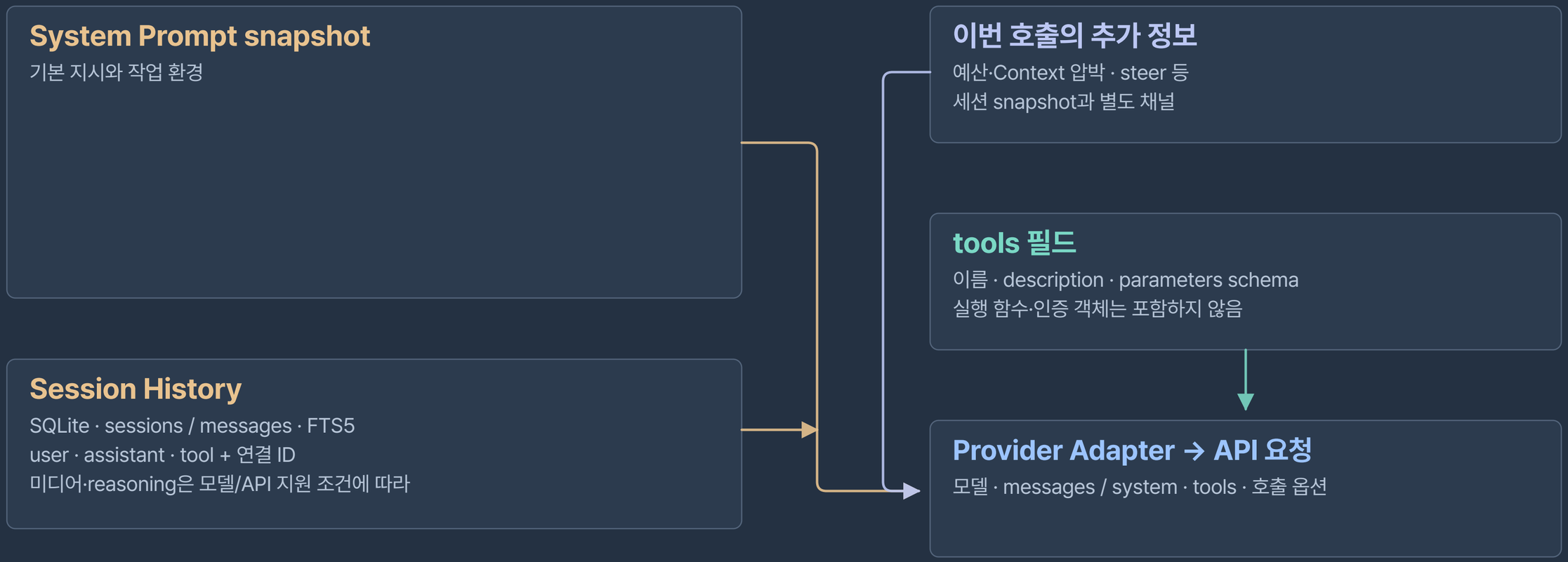
Skills 목록 · MEMORY/USER · 외부 Memory

환경에 따른 정보

Plugin · 프로필 · 시간 · 실행 환경

모델 입력과 Session·Memory의 관계

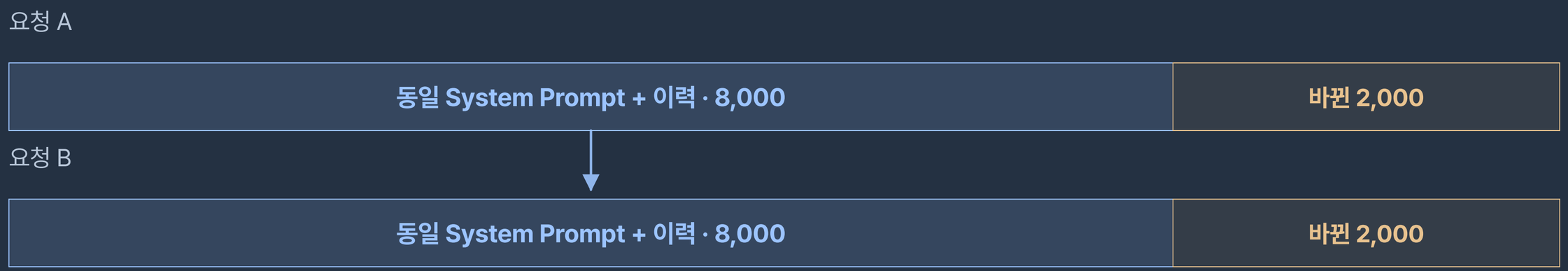
화면의 대화만으로 모델 입력 전체를 설명할 수는 없습니다.



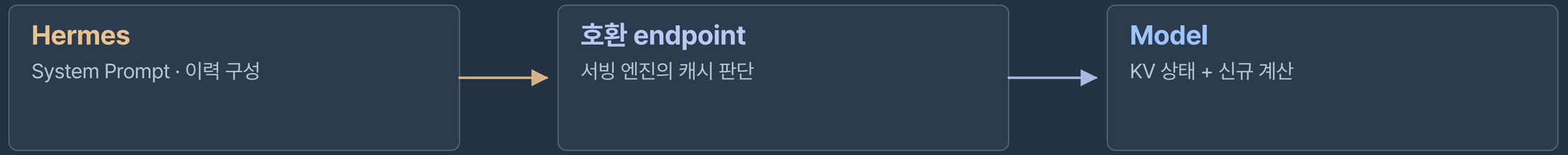
구성 요소의 역할을 확인하고, 실제 payload는 Demo에서 살펴봅니다.

System Prompt를 고정해 Prefix를 재사용하기

같은 System Prompt와 이력의 앞부분을 다시 사용할 수 있습니다.



총 10,000 tokens · 설명용 수치



Hermes는 입력을 구성하고, 캐시 적중은 추론 서비스가 판단합니다.

memory 도구에 무엇을 보낼까?

저장할 대상과 수정할 문장을 지정합니다.

모델이 생성하는 Tool 인자 예시 · Python 공개 API 아님

```
"target": "memory"  # 또는 "user"
"action": "add", "content": "단위를 표시"
"action": "patch", "old_text": "...",
    "new_text": "..."
"action": "remove", "old_text": "..."
"operations": [{"action": "add", ...}]
{"success": true, "done": true, "target": "memory"}
```

파일 선택
MEMORY.md 2,200자 / USER.md 1,375자.
프로필 설정에서 변경 가능.

저장할 작업 지정
add는 추가, patch는 찾아서 수정,
remove는 지정한 내용을 삭제.

여러 변경을 함께 요청
여러 작업은 operations 배열로 전달.
한도 초과 시 오류를 반환하고 쓰지 않음.

현재 대화에서 알 수 있는 값
모델은 자신의 호출 인자와 Tool 응답을
이력으로 봄. 시스템 프롬프트는 그대로.

다음 세션이나 프롬프트를 다시 만드는 시점에 저장된 내용을 읽습니다.

기억을 쓰기 전에 사람에게 물어볼까?

기본은 바로 저장입니다. 승인 설정을 켜면 실행 환경에 따라 달라집니다.

기본 설정

모델이 기억 저장을 요청하면
내용 검사 후 바로 저장

승인을 켰고 사람이 접속 중

지원되는 화면에서
저장할 내용을 보여주고 결정 요청

승인을 켰지만 사람이 없음

예: Cron 실행. 변경 요청을 남겨
나중에 /memory pending에서 확인

write_approval 기본값은 false입니다.

session_search에 무엇을 보낼까?

같은 도구가 인자에 따라 네 가지 동작을 고릅니다.

session_search_tool.py · 실제 인자와 _dispatch 우선순위

```
{"session_id": "abc", "around_message_id": 42,
 "window": 5}
{"session_id": "abc"}
{"query": "", "limit": 3}
{"query": "ALARM-17 OR vacuum", "after": "7d",
 "role_filter": "user,assistant", "limit": 3}
```

① 특정 메시지 주변 보기
세션과 메시지 번호가 있으면 주변 조회.
window 기본 5.

② 세션 열기
세션 ID만 있으면 해당 세션을 읽음.

③ 최근 세션 목록
세션 ID 없이 검색어가 비면 최근 목록.
limit 기본 3, 허용 범위 1~10.

④ 검색어로 찾기
검색어가 있으면 DB에서 검색.
after/before는 세션 시작 시각 기준.

session_id를 보내면 검색어보다 세션 읽기 경로가 먼저 선택됩니다.

검색어 정리와 범위 확인은 누가 할까?

모델의 질의 선택과 Python의 문법·가시성 처리를 구분합니다.

모델이 query 선택
사용자 질문 → Tool 인자

→

코드가 문법 정리
_sanitize_fts5_query

→

SQL과 결과 선택
MATCH · 필터 · 계보 정리

검색어 정리 예
ALARM-17 → "ALARM-17"
따옴표 보존 · 특수 문자·끝 연산자 정리
의미를 새로 추론하는 단계가 아님

조회 범위와 반환
role/source/time 가시성 조건
FTS / CJK / LIKE 대체 경로
선택 메시지와 주변 문맥을 반환

문법상 검색 성공과 업무상 올바른 근거 검색은 따로 확인합니다.

예시 Context가 압축되는 과정

큰 과거 결과를 줄이고, 요약과 최근 문맥을 합칩니다.

압축 전

user: LOT-A의 이상값 확인
tool: 센서 원문 5,000행 …
assistant: 기준선 검토 …
최근 call_7 ↔ result_7

큰 결과 축소

user: LOT-A의 이상값 확인
tool: [큰 과거 본문 축소]
assistant: 기준선 검토 …
최근 call_7 ↔ result_7

요약 + 보호 구간

summary: LOT-A · UTC · SOP r3
원본: artifact://lot-a
최근 call_7 ↔ result_7
최신 사용자 요청 유지

이 예시에서 보호할 정보

정확한 단위·대상·SOP 버전·원본 위치와 아직 사용하지 않은 최근 Tool 결과

실제 문자열·행 수를 재현한 로그가 아닌 압축 원리의 예시입니다.

Tools와 Skills

실행 능력과 재사용 절차를 연결합니다.

도구 등록·검증

승인·실행

Skill 작성·관리

Hermes Tool은 이렇게 등록됩니다

실제 skills_list 등록 코드로 schema와 handler를 구분합니다.

tools/skills_tool.py 731~734의 등록 항목 재배치·축약

```
registry.register(
    name="skills_list",
    toolset="skills",
    schema=SKILLS_LIST_SCHEMA,
    handler=...,  # 실제 lambda 본문 생략
    check_fn=check_skills_requirements
)
```

Discovery
tools/*.py의 AST에서
최상위 registry.register 탐색

Import → Registry
모듈 import 시 등록 실행
schema와 handler를 따로 보관

가용성
활성 Toolset·check_fn
선택한 schema만 모델에 공개

Python 파일을 두는 것만으로 모든 함수가 Tool이 되지는 않습니다.

모델에게 공개되는 Tool의 실제 수준

get_definitions()가 만드는 function schema

실제 SKILLS_LIST_SCHEMA에서 description을 줄인 API 예시

```
{ "type": "function",
  "function": {
    "name": "skills_list",
    "description": "...",
    "parameters": {
      "type": "object",
      "properties": {"category": {"type":"string"}},
      "required": []
    }
  }
}
```

모델에 보이는 것

함수 이름 · 사용 설명
인자 JSON Schema

모델에 안 보내는 것

handler 함수 코드 · check_fn
Python 객체·환경변수 원문

실행할 때

모델은 name + arguments를 제안
Runtime이 handler로 연결

모델은 실행 코드를 받는 것이 아니라 호출 가능한 계약을 받습니다.

Local Tool·MCP·Plugin은 Hermes 어디에 붙을까?

실제 모듈과 실행 위치로 연결 방법을 비교합니다.

Local Tool

tools/*.py → registry.register
model_tools → registry.dispatch

MCP Tool

mcp_tool_discovery → 발견·등록
Client → Transport → 외부 Server

Plugin

register(ctx) → Tool / Hook
Context·Memory·명령 등 확장

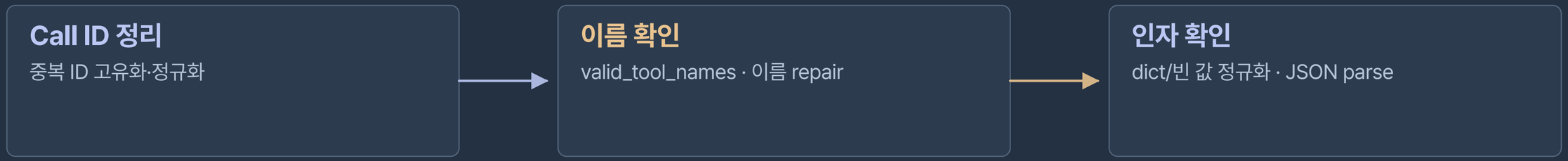
Hermes의 두 실행 경로

memory · session_search · delegate_task 등은 Agent 상태를 쓰는 Inline 경로
일반 Registry/MCP/Plugin 경로와 Hook 적용 범위를 구분

중앙 Hook 하나가 모든 Inline·외부 실행을 감싼다고 가정하지 않습니다.

validate_tool_calls가 검사하는 순서

이름 오류와 JSON 오류는 서로 다르게 처리합니다.



이름 오류

정상·무효 혼합 → 정상 호출 진행, 무효는 오류
모두 무효인 반복 → 3회 후 partial 종료

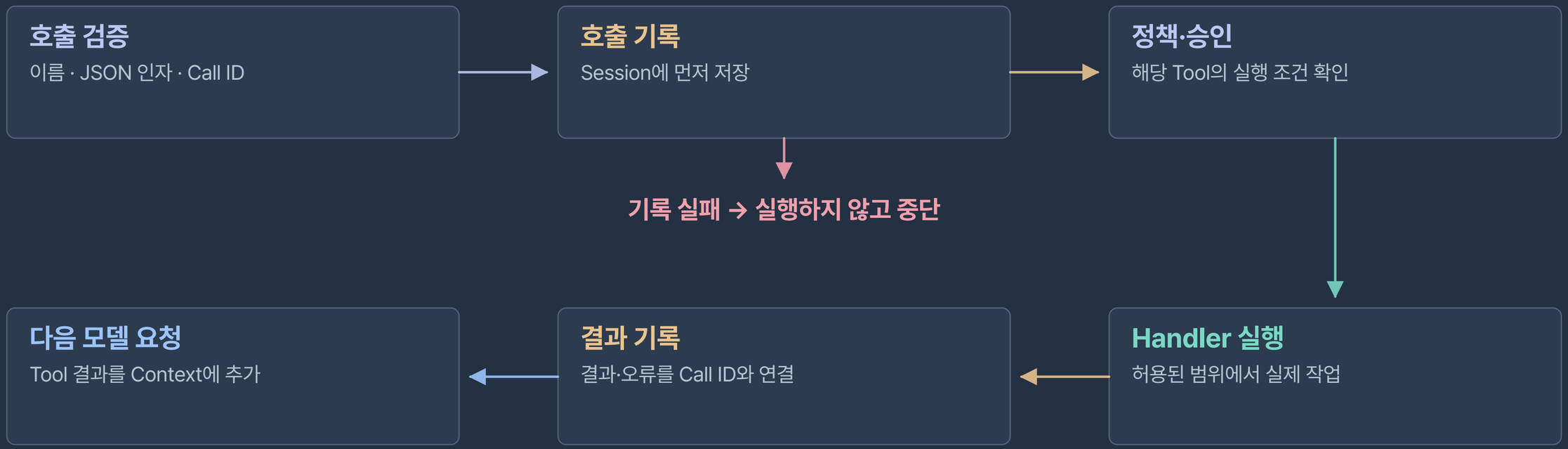
JSON 오류

잘린 인자 → 실행 거부·partial 경로
일반 parse 실패 → 제한 retry 후 Tool 오류 복구

여기서 하는 검사는 모든 Tool의 업무 의미·권한 검증을 대신하지 않습니다.

Tool 요청이 실제 실행으로 이어지는 과정

호출을 검증하고 기록한 뒤, 실행 결과를 다시 모델에 전달합니다.



대표 실행 경로 · Inline Tool과 Backend별 세부 경로는 다를 수 있음

> Tool 실행 전 기록은 필요하지만 외부 작업의 중복 방지까지 보장하지는 않습니다.

실행 전 검사는 실제로 어떤 순서일까?

terminal 명령의 check_all_command_guards 경로

tools/approval.py 1151~1194 · 실제 순서 기반 축약

```
# check_all_command_guards: 분기 축약
if isolated_without_host_access:
    return user_deny_or_approve
if floor_block(command): return deny
if prepared_guard: return prepared_guard
if yolo_or_mode_off: return approve
if permanent_allowlist: return approve
if no_human_surface: return context_policy
if not dangerous or cached: return approve
return human_decision(..., smart=smart_mode)
```

명령 실행 경로

terminal 등의 사전 guard
모든 Tool 공통 함수는 아님

모드보다 앞선 검사

host 접근·강제 floor
준비된 승인 확인

사람에게 묻기 전

allowlist·cached 승인
무인 policy·위험 패턴 검사

off·격리 실행도 모든 Tool·권한 정책을 전부 해제한다는 뜻은 아닙니다.

어떤 경우에 실제 사용자 승인을 요청할까?

위험 판정·기존 승인·실행 표면을 확인한 뒤 요청합니다.

위험 패턴 감지
명령·reason·pattern key 생성

→

smart / manual
smart는 보조 판단 경로
manual은 사람 검토 경로

→

사용자 응답
once · session · always · deny
표면·정책에 따라 선택 제한

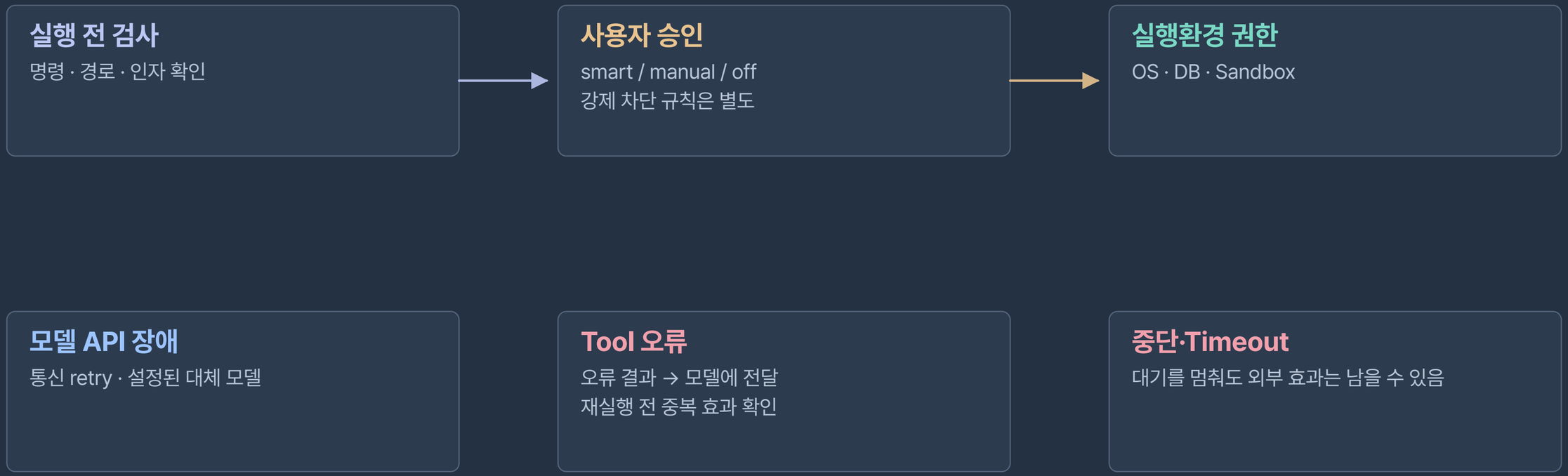
smart 판단
APPROVE / DENY / ESCALATE
실패·불확실한 응답은 escalate

사람이 응답할 수 없을 때
Cron·single-query·unattended 별도 정책
기본 deny · 무한 승인 대기로 남기지 않음

smart가 모든 요청을 사람에게 묻는 모드인 것은 아닙니다.

오류·취소가 실제 작업에 남기는 것

모델 요청 실패와 이미 실행한 Tool의 효과를 구분합니다.



Checkpoint: opt-in 파일 복구 · 외부 API·DB 변경 전체의 rollback은 아님

> 기록 실패·중단·외부 효과 미상을 같은 성공 상태로 합치지 않습니다.

Hermes의 병렬 Tool 실행 코드

계획한 구간 안에서 실제 worker를 스케줄링합니다.

agent/tool_executor.py · 공식 코드 발췌/줄바꿈 조정

```
# _ConcurrentBatch.submit_all 실제 호출 형태
f = executor.submit(
    propagate_context_to_thread(self.run_worker),
    i, submit_index
)

# 원본은 한 줄, 화면에서 줄바꿈 조정
# self.results[index] = outcome
# 결과 slot은 원래 호출 index를 유지
```

구간 계획

_plan_tool_batch_segments
경로 충돌·interactive barrier

Worker 실행

DaemonThreadPoolExecutor
Context·승인 callback 전달

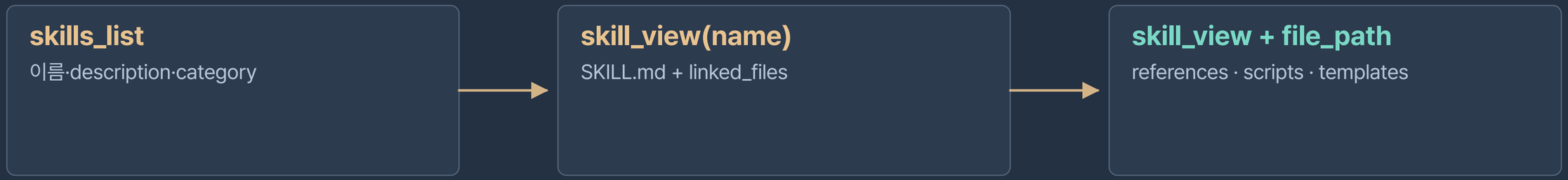
결과 정리

self.results[index]에 보관
완료 순서와 호출 순서를 분리

thread 여러 개와 독립 Agent 여러 개는 다른 실행 방식입니다.

Skills를 찾고 읽는 Hermes의 경로

목록·본문·참고 자료와 명시·자동 로딩을 구분합니다.



명시 /skill-name · auto_load · 모델 선택 · Cron 첨부

모든 설치 Skill이 매번 본문 전체로 들어가는 것은 아닙니다.

> Skill은 실행 함수가 아니라 도구 사용 절차를 담은 재사용 문서입니다.

Hermes는 언제 새 Skill을 만들까?

지침이 기록을 유도하고 모델이 실제 Tool Call을 선택합니다.

학습할 경험 발생
새 절차 · 반복 시행착오
사용자 수정·피드백

→

모델이 재사용 판단
System guidance · 사용자 /learn
주기적 nudge·review 유도

→

skill_manage 요청
실제 파일 변경 Tool 호출
지시만으로 자동 저장되는 것은 아님

판단과 검증은 다른 단계
성공했다는 이유만으로 검증된 공식 절차가 되는 것은 아님 · 다음 장에서 쓰기 경로 확인

자기 개선은 재사용 절차의 갱신이며 모델 가중치 학습이 아닙니다.

skill_manage의 변경은 어떻게 반영될까?

파일 검증·쓰기 승인·보안 검사·변경 이력을 구분합니다.

Tool 요청 예시와 v0.22.5 기본 설정은 구분해서 표시

```
skill_manage(
  operations=[{
    "action": "create",
    "name": "alarm-review",
    "content": "...SKILL.md 본문..."
  }]
)

skills:
  write_approval: false
  guard_agent_created: false
  ledger: true
```

검증·변경
경로·동작·대상 확인
create / patch / write_file / delete

승인을 켜면
Skill 쓰기는 pending으로 stage
/skills diff → approve/reject

관리 기록
before/after hash ledger
Curator backup·rollback과 연결

내용 검사와 사람의 승인, 업무 정확성 검증은 서로 다른 절차입니다.

Curator는 언제 무엇을 정리할까?

일정·유휴 조건을 만족하면 사용 이력으로 상태를 바꿉니다.

실행 여부 확인

enabled=true · interval 168h
idle 2h 이상이 기본 조건

사용 이력 확인

view / use / patch 기록
pin·Cron 참조·대상 범위 확인

자동 상태 전이

14일 미사용 → stale
30일 미사용 → archive

기본 Curator는 LLM 없는 결정적 정리

Agent 생성 대상 중심 · Hub 설치 제외 · 기본 자동 삭제 없음 · 첫 관찰은 실행 시각 초기화

active → stale → archived는 사용 이력 관리이며 품질 점수 판정이 아닙니다.

Curator의 LLM 통합은 별도 선택입니다

기본 정리와 비용이 드는 절차 통합을 나누어 봅니다.

v0.22.5 기본 설정

```yaml
curator:
  enabled: true
  interval_hours: 168
  min_idle_hours: 2
  stale_after_days: 14
  archive_after_days: 30
  consolidate: false
  prune_builtins: false
```

선택적 LLM 검토

consolidate=true 또는 수동 실행
별도 AIAgent가 겹침·drift 검토

허용된 변경

keep · patch · 통합 · archive
지원 파일과 참조 관계도 유지

직접 점검

hermes curator status
hermes curator run --dry-run

Skill을 합칠 때 본문뿐 아니라 references·scripts·템플릿도 함께 관리합니다.

안전장치와 실행 기록

실행을 제한하는 판단과 실행을 설명하는 기록을 연결합니다.

기본 보호 장치

반복·정책 제어

Trace 수집·활용

Hermes의 기본 안전장치는 어디에 있을까?

명령 승인 외에도 서로 다른 경계에서 실행을 제한합니다.

실행 통제

명령의 floor·smart/manual 승인
Tool 범위·Hook·반복 감지

파일·입력 보호

민감 경로·지침 파일 쓰기 검사
URL 제한·비밀값 마스킹

격리와 복구

terminal.backend 기본 local
Checkpoint는 기본 off

하나의 만능 Guardrail이 있는 구조는 아닙니다

승인·파일 검사·OS 권한·반복 감지는 적용 위치와 실패 방식이 다릅니다.

기본 local 실행의 권한과 격리 환경의 권한을 구분해 설정합니다.

반복되는 Tool 호출을 코드에서 감지하기

한도 검사와 실행 결과 관찰이 서로 다른 시점에 동작합니다.

tool_guardrails.py + tool_executor.py · 변수/래퍼 축약

```
# 실제 Controller 메서드를 연결한 축약
d = agent._tool_guardrails.before_call(name, args)
if d.allows_execution:
    result = execute(args)
    after = agent._tool_guardrails.after_call(
        name, args, result
    )
    seen = agent._tool_guardrails.observe_call(...)
# warn: 결과에 안내 추가
# block/halt: Runtime이 실행/반복 중단 처리
```

호출 식별

Tool 이름 + canonical args → hash
턴마다 실패·반복 카운터 초기화

실행 전

before_call: cap·이전 실패 확인
허용 / 차단 결정

실행 후

after_call + observe_call
실패·동일 결과·순환 패턴 관찰

실패를 몇 번 했는지뿐 아니라 그 사이에 진전이 있었는지를 봅니다.

기본 경고와 강제 중단의 차이

대화형 세션과 무인 작업의 기본 정책이 다릅니다.

DEFAULT_CONFIG · v0.22.5

```yaml
tool_loop_guardrails:
  warnings_enabled: true
  hard_stop_enabled: false
  non_interactive_hard_stop_enabled: true
  warn_after:
    exact_failure: 2
    same_tool_failure: 3
    idempotent_no_progress: 2
  hard_stop_after:
    exact_failure: 5
    same_tool_failure: 8
    idempotent_no_progress: 5
```

대화형 기본

경고를 결과에 붙여 복구 유도
반복 감지 hard stop은 off

무인 표면

해당 platform에서 hard stop on
Cron 등 사람이 없는 반복 제어

항상 별도 적용

web_search 50 · subagent 50 / turn
loop_caps는 hard_stop과 별개

threshold는 실패 분류·진전·표면 조건과 함께 해석합니다.

필요한 실행 정책을 Hook으로 추가하기

공식 pre_tool_call의 반환값과 실패 처리를 확인합니다.

공식 Hook API 활용 예시 · terminal만 차단하는 제한된 예

```
# 로컬 Plugin의 정책 예시
def register(ctx):
    ctx.register_hook("pre_tool_call", read_only)

def read_only(**event):
    if event.get("tool_name") == "terminal":
        return {
            "action": "block",
            "message": "조회 전용 세션"
        }
```

정책 Hook

block 또는 modify 인자 반환
효과를 바꾸는 확장 지점

실패 방식

pre_tool_call 오류·시간 초과 → 차단
일반 관측 Hook은 실패 시 건너뜀

활용 범위

tool_request 수정 → 정책·승인
최종 인자로 downstream 검사

이 예시는 terminal 차단이며 모든 파일·외부 쓰기를 막는 완전한 조회 모드는 아닙니다.

Hermes에 기본으로 남는 기록은 무엇일까?

대화 저장·운영 로그·이벤트 관측·외부 전송을 구분합니다.

Session 기록	SQLite · messages · Tool Call/Result 연결
운영 로그	logs/agent.log INFO+ · errors.log WARNING+
Observer / Span	API·Tool·승인·자식 이벤트를 ID로 연결
별도 내보내기	Trajectory / MoA trace / HF upload / OTLP는 설정·목적별 구분

> Session 기록만으로 모든 API 시도의 지연·재시도를 복원할 수는 없습니다.

Observer Hook으로 실행 경계를 연결하기

이벤트 종류와 상관 ID가 Trace의 뼈대가 됩니다.

모델 요청 경계

pre_api_request
post_api_request / api_request_error

Tool 경계

pre_tool_call
post_tool_call: status·duration_ms

사용자·자식 경계

approval request / response
subagent_start / stop

이벤트를 연결하는 ID

session_id · turn_id · api_request_id · tool_call_id
parent/child_session_id · parent_turn_id로 위임 관계 연결

발생 순서만 보고 연결하지 않고 명시적인 ID와 시간 정보를 사용합니다.

짧은 Plugin으로 API 관측을 시작하기

먼저 식별자와 지연을 수집하고 필요한 필드만 확장합니다.

공식 등록 API를 이용한 최소 관측 Plugin 예시

```python
import logging
log = logging.getLogger("seminar.trace")

def register(ctx):
    ctx.register_hook("post_api_request", observe)
    ctx.register_hook("api_request_error", observe)

def observe(**e):
    log.info("turn=%s request=%s duration=%s",
        e.get("turn_id"), e.get("api_request_id"),
        e.get("api_duration"))
```

수집 시작

Plugin 발견·활성화 후 register
기본 내장 Exporter 코드가 아님

payload 가공

JSON 변환 · key redaction · 크기 제한
소비자가 있을 때 구성

수집 품질

빠른 callback·비동기/배치 Export
누락·닫히지 않은 Span 점검

관측 callback의 성공 여부가 작업 성공 여부와 같지는 않습니다.

Trace로 무엇을 확인하고 개선할까?

재시도·차단·사용량·위임을 한 작업의 기록으로 읽습니다.

수집 데이터에서 확인할 관계의 예시

```
# 가상 이벤트 · 성능 측정 결과 아님
turn=T1 request=R1 error=429 retry_count=0
turn=T1 request=R1 ok usage={...}
turn=T1 tool=C1 status=blocked duration_ms=...
parent_session=S1 child_session=S2

# 같은 R1이라고 같은 HTTP 시도는 아님
# 시간 + retry_count + 수집 측 attempt로 구분
```

실패 원인

모델/API 오류 vs 정책 차단
Tool 실패·취소·전달 실패 분리

비용·지연

실제 usage·API duration 집계
승인 대기·Tool·자식 시간 구분

Demo 활용

질문 → 요청 → 승인/Tool → 저장
같은 ID로 Sources 설명과 연결

실행 완료·정책 통과·업무 정답 여부를 별도의 기준으로 평가합니다.

실행 대화를 Trace Viewer로 공유하는 경로

저장된 대화를 재구성하는 기능과 실시간 계측을 구분합니다.

명시적 내보내기

/upload-trace
Session 메시지 조회

JSONL 변환

system 제외 · tool_use/tool_result
uuid / parentUuid로 순서 연결

Hugging Face 업로드

기본 private=true · redact=true
토큰·패키지·명시 호출 필요

Transcript Viewer와 실제 시간 Trace의 차이

현재 변환 코드는 내보내기 시각을 timestamp로 사용 · 실제 API span latency 복원용이 아님

내보내기 전 원문·비밀값·공유 범위를 검토합니다.

위임

부모의 요청 → 자식 실행 → 결과 회수

delegate_task는 자식을 어떻게 만들까?

작업 요청을 정규화하고 Child AIAgent를 구성합니다.

실제 delegate_task schema를 따른 예시 · 실행 기록 아님

```
delegate_task(
  tasks=[
    {"goal": "측정값 검토",
     "context": "합성 LOT-A / UTC / 기준 v3"},
    {"goal": "SOP 근거 검토",
     "context": "승인 문서 r3만 확인"}
  ]
)
# 모델의 Tool 요청 예시
```

인자 정규화

goal 또는 tasks를 작업 목록으로
할당량·권한·동시성 확인

자식 구성

독립 AIAgent·Messages·Task
부모 기반 Toolset·모델 설정

Parent 범위

goal/context로 필요한 근거 전달
부모 History 전체 자동 상속 아님

자식이 알아야 할 데이터·버전·작업 범위를 context에 명시합니다.

자식 실행과 결과 전달의 세부 경로

대기형 반환과 background handle을 구분합니다.

_run_batch
동기 / background 경로 선택

→

_run_children_parallel
Child 실행과 진행·중단 추적

→

결과 전달
summary · status · schema 검사
Parent가 다음 판단 수행

최상위 background 경로
즉시 handle / delegation_id 반환
완료 결과는 별도 전달로 도착

output_schema를 지정하면
JSON Schema 확인 → 1회 수정 기회
실패해도 미검증 표시와 원문 결과 유지

Child 완료와 결과의 구조·업무 정확성 통과는 별개입니다.

Hermes는 delegate_task만 사용할까?

공식 소스·문서에는 목적이 다른 다중 실행 방식이 있습니다.

delegate_task

하위 목표마다 별도 Loop
Task·도구·예산을 나눠 실행

Mixture of Agents

Provider의 reference + aggregator
acting model이 Tool Loop 수행

Kanban

Board·Task·Profile·Worker로 조정
지속 작업 상태와 협업

선택 기준은 분리하려는 단위

독립 하위 작업 / 여러 모델의 관점 / 지속 작업 보드 · 서로 다른 구조와 수명

작업 위임과 다중 모델 합성, 보드 기반 협업을 구분합니다.

예약 실행과 활용

반복 작업을 예약하고 실행 상태를 직접 다룹니다.

Cron 구조

생성·관리 명령

Session·Skill 활용

Cron으로 예약 작업 실행하기

예약 시각이 되면 새 Agent 작업을 시작하고 결과를 전달합니다.



구분해서 확인할 세 가지

예약 중복 방지
같은 예약 회차의 시작 관리

외부 작업
중복 부작용은 대상 시스템과 확인

결과 전달
작업 성공과 전달 성공을 구분

Agent 작업 모드의 예시 · 새 Session도 지속 Memory와 Skills를 사용할 수 있음

> 실행 슬롯의 중복 방지는 외부 작업의 exactly-once 보장과 다릅니다.

Cron 작업을 만들고 관리하기

자연어·slash command·독립 CLI가 같은 예약 기능으로 이어집니다.

공식 문법을 따른 예시 · 실제 예약을 생성하지 않음

```
# 대화 안의 slash command
/cron add "every 2h" "합성 로그 요약"

# 독립 CLI
hermes cron create "every 2h" "합성 로그 요약"
hermes cron list

# 자연어로도 예약 요청 가능
# 내부 Tool: cronjob_manage
```

생성

schedule + prompt
Skill·실행 환경·전달 대상 확인

관리

list · pause · resume · run
update · remove 등

실행 시

설정 검증 → 새 Session
첨부 Skill·prompt → Loop

schedule 표현과 prompt, 권한·전달 대상을 함께 확인합니다.

Cron은 언제 유용하고 무엇을 주의할까?

정기 점검·요약·알림에 맞는 기능과 운영 비용을 함께 봅니다.

장점

시간·주기 기반 작업 시작
Prompt + Skill로 절차 재사용
Fresh Session으로 이력 누적 완화
결과 저장·전달을 작업과 연결

주의할 점

무인 승인·인증 만료·모델 비용
실행 성공과 전달 성공의 차이
외부 작업 중복 효과 별도 통제
중단 후 재실행·결과 확인 필요

Agent 모드: LLM·Tools 실행 / no-agent 모드: Script 결과 전달

Script의 실행 권한과 LLM Tool 승인 정책을 자동으로 동일시하지 않습니다.

> 예약 회차 Claim이 외부 작업과 전달까지 exactly-once로 만들지는 않습니다.

구조를 알고 쓰면 유용한 Hermes 명령

세션 경계와 Skill 관리가 실제 사용 습관으로 이어집니다.

/new

새 세션 · 새 snapshot
작업/주제가 바뀔 때

/learn ...

경험·자료를 Skill로 정리
생성 내용은 검토 후 재사용

/skills diff <id>

보류된 변경을 확인
write_approval=true일 때 검토

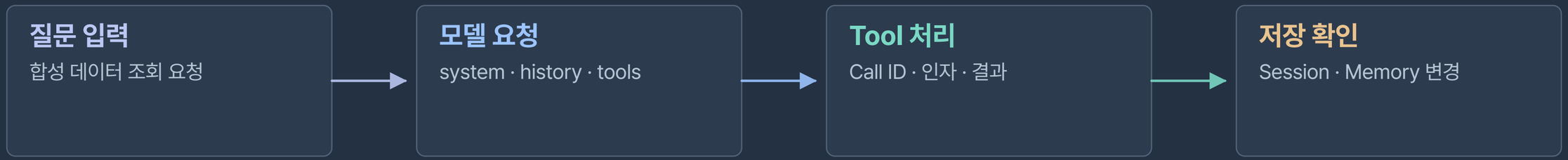
명령 뒤의 구조를 함께 보기

세션 경계 / 절차 파일 작성 / pending 검토 · 서로 다른 상태를 바꾸는 입구

명령의 이름보다 무엇이 바뀌고 언제 반영되는지를 확인합니다.

Hermes Demo

지금까지 설명한 요청·Tool·저장 과정을 실행 기록에서 확인합니다.



후속 질문과 Memory

누적 이력 · 파일 diff
현재 snapshot과 다음 세션 비교

선택한 확장 한 경로

Skill · MCP · 위임 중 실제 발생한 경로
나머지는 별도 실행 기록으로 확인

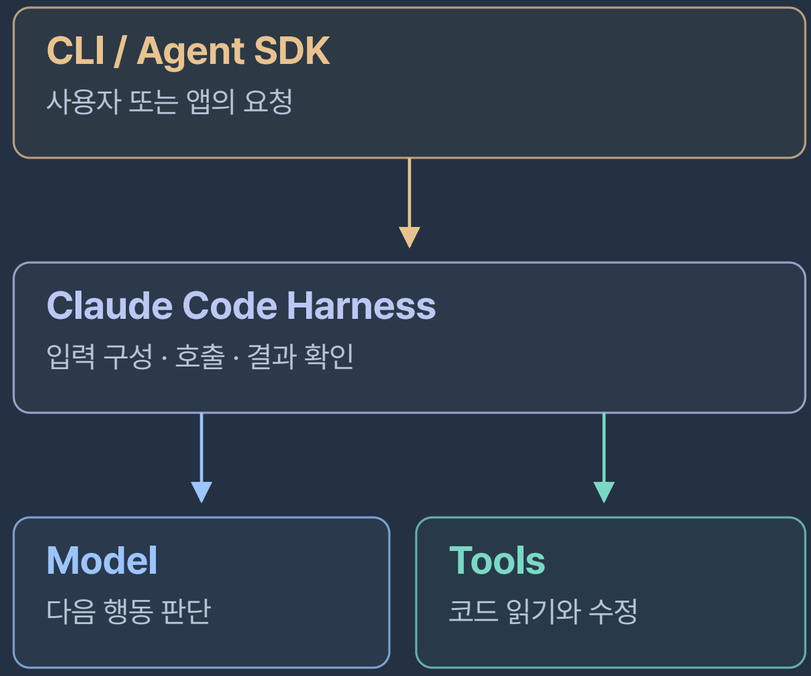
현재 Demo 링크는 연결 안내입니다. 실제 실행 서비스는 별도로 연결합니다.

> 기본 흐름을 먼저 확인하고, 확장 기능은 실제 발생한 경로만 살펴봅니다.

Claude Code

코딩 작업을 위한 Agent Harness

Context · 실행 정책 · 위임



코딩 작업은 어떻게 진행될까?

정보를 모으고, 도구를 쓰고, 결과를 확인합니다.



결과가 더 필요하면 정보를 다시 모읍니다.

CLAUDE CODE

Context와 기억

모델에 들어가는 입력과 다음 세션에 남는 기록

01 요청의 세 계층

02 지침 로딩

03 Auto Memory

모델 입력의 세 계층

고정 지침 위에 프로젝트 정보와 대화가 쌓입니다.

계층	입력 예시	들어오는 시점
System	제품 지침과 Tool 정의	세션 및 실행 구성에 따라 준비
Project	프로젝트 지침과 환경 정보	시작 또는 관련 경로 접근
Conversation	질문, 응답, 도구 결과	턴이 진행되며 누적

세 계층은 요청을 이해하기 위한 구분입니다.

어떤 지침을 언제 읽을까?

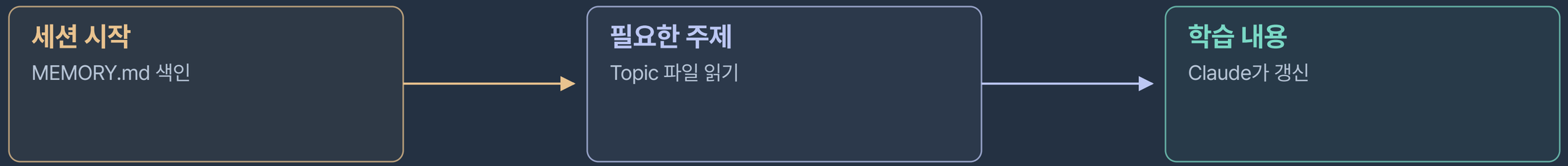
파일 위치와 적용 조건이 로딩 시점을 결정합니다.

위치	읽는 시점	확인할 조건
CLAUDE.md	사용자·프로젝트 지침은 시작 시	하위 폴더 지침은 관련 파일 접근 시
.claude/rules/	전역 또는 일치하는 경로 작업 시	frontmatter의 paths 설정
AGENTS.md	직접 로딩 또는 명시적 import	CLI 버전과 지침 파일 선택 설정

/memory에서 현재 적용 중인 지침을 확인할 수 있습니다.

Auto Memory는 색인부터 읽습니다

자주 쓰는 내용은 MEMORY.md에, 상세 기록은 Topic 파일에 둡니다.



시작 시 색인의 앞 200줄 또는 25KB까지 읽습니다.

CLAUDE CODE

Tools와 Skills

도구 명세와 작업 절차를 필요한 때 불러옵니다.

01 Tool Search

02 Skill 호출

03 Plugin 배포

필요한 MCP Tool의 명세를 불러오기

ToolSearch로 작업에 맞는 명세를 요청합니다.



호환 모델에서는 지연 로딩이 기본이며 auto는 임계값 모드입니다.

Skill 본문과 권한의 수명

본문이 대화에 남는 동안에도 사전 승인은 호출 턴에만 적용됩니다.

Skill frontmatter 예시

```
---
name: review-changes
description: Review changed files
allowed-tools: Read, Grep
---
```

호출
사용자 명령 또는 모델 선택

본문
호출 뒤 대화에 남을 수 있음

사전 승인
지정 Tool에 호출 턴 동안 적용

도구 범위를 제한할 때는 권한 규칙도 확인합니다.

Skill fork와 대화 fork의 입력

자식에게 무엇을 전달하는지 비교합니다.

경로	자식의 시작 입력	결과의 행선지
Skill context: fork	Skill 본문이 작업 입력	호출자에게 결과 전달
대화 fork	현재 대화에서 분기	분기한 대화에서 계속 작업
Subagent의 skills	지정 Skill을 시작에 로드	역할 지침과 함께 자식에서 사용

어느 쪽에서 Skill을 지정하는지에 따라 로딩 방향이 달라집니다.

Plugin으로 작업 환경을 배포하기

프로젝트마다 쓰던 확장 구성을 한 묶음으로 공유합니다.

Plugin 디렉터리 예시

```
review-kit/
  .claude-plugin/plugin.json
  skills/review/SKILL.md
  agents/reviewer.md
  hooks/hooks.json
  .mcp.json
```

Manifest

이름과 구성 위치

실행 구성

Skills · Agents · Hooks · MCP

Hermes의 register(ctx)와 Claude Plugin은 확장 접점이 다릅니다.

CLAUDE CODE

안전장치와 실행 기록

Tool을 실행하기 전의 판단과 실행 후에 남는 기록

01 권한과 Hook

02 Bash Sandbox

03 OTel과 Trace

권한 규칙은 Tool 호출에 적용됩니다

같은 Bash 호출도 규칙과 모드에 따라 결과가 달라집니다.

settings.json 일부

```
{
  "permissions": {
    "deny": ["Bash(rm *)"],
    "ask": ["Bash(git push *)"],
    "allow": ["Bash(git status *)"]
  }
}
```

deny
일치하면 호출 차단

ask
승인 필요

allow
일치하는 호출 허용

규칙은 deny, ask, allow 순으로 확인합니다.

PreToolUse가 받는 입력과 돌려주는 결정

Tool 이름과 인자를 확인한 뒤 실행 전에 판단합니다.

Command hook 응답 JSON 예시

```
{
  "hookSpecificOutput": {
    "hookEventName": "PreToolUse",
    "permissionDecision": "deny",
    "permissionDecisionReason":
      "Protected path"
  }
}
```

matcher
이벤트가 적용될 Tool 선택

tool_input
명령 또는 파일 인자 확인

updatedInput
필요 시 실행 인자 수정

PostToolUse는 실행 결과를 받고 Stop은 응답 종료에 개입합니다.

Hook이 실패하면 실행은 계속될까?

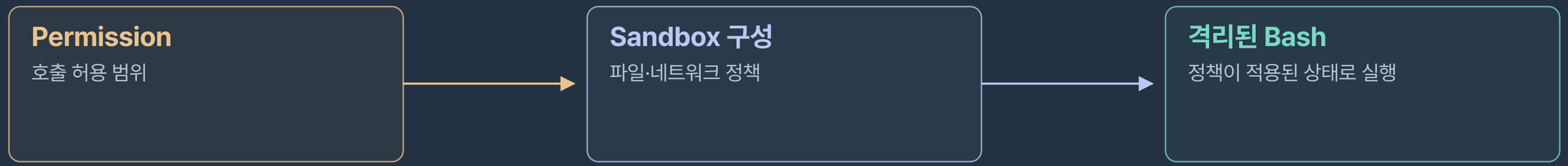
정책 거부와 handler 오류는 결과가 다릅니다.

상황	일반 PreToolUse 처리	확인할 예외
명시 deny / exit 2	호출을 차단	구조화 결정과 종료 코드를 구분
일반 오류	통상 기존 권한 흐름으로 진행	command/http onFailure: block
timeout	통상 결정 없이 권한 흐름으로	SDK callback timeout은 차단
onFailure: block	지원 조건에서 실패도 차단	네이티브 Windows 예외 확인

정책용 Hook은 오류와 timeout까지 별도로 시험합니다.

Bash Sandbox가 제한하는 범위

권한 판정 뒤 Shell 프로세스의 파일과 네트워크 접근을 제한합니다.



파일 Tool, MCP 서버, Hook은 각각의 실행 경계를 확인합니다.

지표, 이벤트, Trace는 무엇을 남길까?

기록 목적에 맞춰 수집과 내보내기를 설정합니다.

Console metrics 설정 예시

```
export CLAUDE_CODE_ENABLE_TELEMETRY=1
export OTEL_METRICS_EXPORTER=console
```

Metrics
세션 수, 토큰, 비용의 집계

Events
Tool 결정과 실행 같은 사건

Trace
한 요청에 속한 작업과 시간

기본 비활성인 Trace는 별도의 활성화 설정을 확인합니다.

승인 대기와 실행 시간을 나눠 보기

Tool span 안에서 사람이 기다린 시간과 실제 실행 시간을 읽습니다.

공식 span 관계의 축약도

```
claude_code.interaction
  +-- claude_code.llm_request
  +-- claude_code.hook  (detailed beta)
  +-- claude_code.tool
        +-- blocked_on_user
        +-- execution
        +-- child agent spans
```

모델 실패
llm_request 상태 확인

승인과 실행
대기와 실행 구간 분리

상관관계
trace_id와 span_id 연결

입력과 출력 내용의 수집 범위는 별도 설정으로 결정합니다.

CLAUDE CODE

긴 작업과 위임

길어진 대화는 정리하고 독립된 작업은 나눕니다.

01 캐시와 압축

02 Subagent와 Teams

03 격리와 복구

변경이 캐시에 미치는 범위

재사용할 수 있는 입력과 근거의 최신성을 따로 확인합니다.

변경 예시	캐시에서 볼 부분	업무에서 볼 부분
모델·Tool 구성	앞쪽 요청 구성의 변경	도구 가용성과 모델 호환성
파일·Skill 갱신	변경 내용이 입력된 위치	최신 파일을 다시 읽었는가
대화 압축	Conversation 입력이 재구성	근거와 제약이 요약에 남았는가

캐시 적중은 같은 입력의 재사용을 보여 줍니다.

압축 뒤에는 무엇을 다시 읽을까?

대화 요약과 재주입되는 지침을 분리해 봅니다.



중요 파일과 경로 규칙은 다음 작업에서 다시 확인합니다.

Subagent의 역할을 파일로 정의하기

역할 설명과 실행에 필요한 도구를 선택합니다.

.claude/agents/reviewer.md 예시

```
---
name: reviewer
description: Review changed files
tools: Read, Grep, Glob
model: inherit
---
Check correctness and report findings.
```

description
위임 대상을 고를 때 사용

본문
자식의 역할 지침

설정
도구·모델·권한·기억 범위

부모는 작업을 전달하고 자식의 결과를 받습니다.

새 Context, 대화 fork, background

입력의 범위와 결과를 기다리는 방식은 독립된 선택입니다.

선택	입력과 실행	호출자에게 돌아오는 방식
전문 Subagent	역할 지침과 위임 작업	결과 보고
대화 fork	기존 대화에서 분기	분기한 작업의 결과
Background	실행을 맡기고 다른 작업 진행	완료 알림 또는 결과 회수

중첩 위임은 자식의 도구와 실행 환경 조건을 따릅니다.

Teams는 작업 목록과 메시지로 협업합니다

Lead와 여러 독립 세션이 작업 상태를 공유합니다.

요소	팀에서 맡는 일	공유 범위
Lead	작업을 만들고 진행을 조정	팀 구성과 작업 목록
Teammate	별도 Context에서 할당 작업 수행	작업 상태와 팀 메시지
Task list	할당, 의존성, 완료 상태 관리	팀 전체가 참조

Agent Teams는 실험 기능이며 기본 비활성입니다.

대화와 파일을 각각 복구하기

격리하는 대상과 되돌릴 수 있는 대상을 확인합니다.

기능	주로 관리하는 상태	적용 범위
resume / fork	대화 이력	이어서 진행하거나 대화를 분기
Worktree	작업 디렉터리	다른 Git 작업 트리에서 편집
Checkpoint / rewind	추적된 파일 변경과 대화	선택한 시점과 복원 대상을 지정

Shell과 외부 서비스의 부작용은 별도의 복구 절차가 필요합니다.

CLAUDE CODE

예약과 활용

작업의 실행 위치와 일상적인 사용 흐름을 정합니다.

01 예약의 수명

02 작업별 명령

03 Harness 선택

예약 작업은 어디에 남을까?

등록 정보의 수명과 실제 실행에 필요한 호스트를 구분합니다.

경로	실행 위치	실행 조건
Session / /loop	현재 Claude Code 세션	세션 실행 및 유효 기간
Desktop	설정한 로컬 환경	앱과 호스트의 상태
Cloud	설정한 클라우드 환경	환경·연결·예약 설정

예시 /loop 10m 열린 PR의 상태를 확인해 줘

탐색, 실행, 검토에 필요한 명령

코드 검토를 시작해 결과를 정리하는 순서로 살펴봅니다.

시점	명령	확인할 내용
탐색	/context · /memory	입력 사용량과 적용 중인 지침
실행	/permissions · /hooks	허용 규칙과 실행 전후 검사
검토	/compact · /resume · /rewind	대화 정리, 이어가기, 복구

작업에 필요한 명령을 실행 구조와 연결해 기억합니다.

두 Harness에서 어디까지 바꿀 수 있을까?

바꾸려는 동작에 따라 수정할 계층을 고릅니다.

변경 요구	Hermes	Claude Code
업무 지침과 절차	프로젝트 지침·Skills	CLAUDE.md·Rules·Skills
도구와 실행 정책	Plugin·Tool·Backend	MCP·Plugin·권한·Hook
실행기 자체	공개 코어를 수정할 수 있음	공개 설정·확장 계약으로 통합

이제 PydanticAI에서 개발자가 구성하는 실행 경로를 살펴봅니다.

PydanticAI



코드로 구성하는 Agent 실행과 검증

Agent 정의

내부 실행 Graph

업무 검증과 확장

PYDANTIC AI / 01

Python 코드로 Agent 구성하기

작은 LOT 조회 Agent에서 시작합니다.

Pydantic과 PydanticAI, 어디까지 맡을까?

같은 불량률 보고서를 만드는 코드에서 역할을 나눠 봅니다.

Pydantic · 데이터 검증

Report에서 bin_id와 defect_rate 타입 확인
문자열·숫자 등 입력을 선언된 계약으로 검증

PydanticAI Core · 실행

모델 호출 → 도구 인자 검증 → Python 실행
결과를 다음 메시지로 넣거나 최종 타입으로 반환

Harness · 선택 기능

Memory, Skills, Subagents를 연결
저장소나 호출 범위는 앱 설정으로 선택

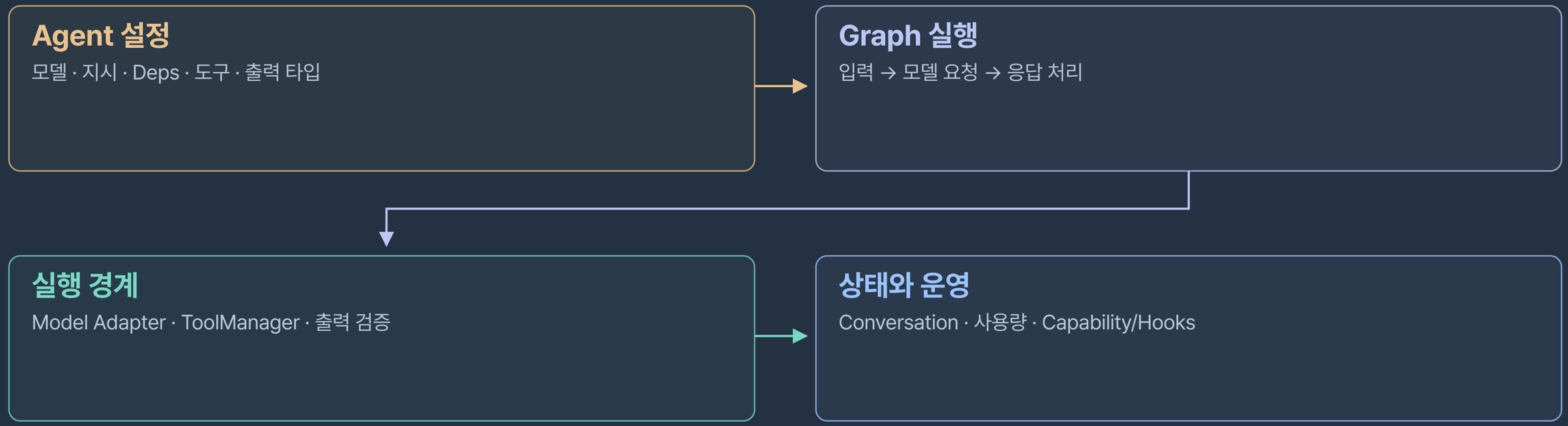
업무 앱 · 실제 기준

조회 가능한 BIN인지, 42/1000이 맞는지
데이터를 다시 조회하고 승인·저장을 관리

형식이 맞는 보고서와 수치가 맞는 보고서를 각각 확인해야 합니다.

PydanticAI Core의 네 부분

Agent 선언을 시작으로 실행, 모델·도구, 대화와 확장을 순서대로 봅니다.



Harness는 Core의 Capability 경계에 필요한 기능을 연결합니다.

가장 작은 Agent 실행

모델을 정하고 요청을 보내면 Core가 결과를 돌려줍니다.

앱 코드 · 로컬 테스트 모델

```
from pydantic_ai import Agent
from pydantic_ai.models.test import TestModel

agent = Agent(TestModel())
result = agent.run_sync('LOT-A 조회 준비')
print(result.output)
```

TestModel로 연결 확인

run_sync에서 결과 받기

모델을 정하고 요청을 보내면 Core가 결과를 돌려줍니다.

조회 도구와 보고서 타입을 붙이기

앱에서 선언한 세 가지가 실행 중 어디에 쓰이는지 봅니다.

Python 앱 코드 발췌 · Repo와 모델 연결 정의 생략

```python
class YieldReport(BaseModel):
    lot_id: str
    yield_pct: float
```

```python
agent = Agent(model, deps_type=Repo,
              output_type=YieldReport)
```

```python
@agent.tool
def read_counts(ctx: RunContext[Repo], lot_id: str):
    return ctx.deps.read_counts(lot_id)
```

```python
result = agent.run_sync("LOT-A 조회", deps=repo)
report = result.output
```

① 결과 타입
최종 결과의 형태를 정함.
수치가 실제 조회값과 같은지는 별도 검사.

② Agent 설정
Repo는 앱에서 넣을 의존성 타입.
YieldReport는 모델 출력 계약.

③ 조회 함수 등록
모델에는 lot_id 인자만 공개.
함수는 서버에서 ctx.deps로 조회.

④ 실행
조회와 출력 검증을 마치면
앱이 YieldReport 객체를 받음.

개발자는 데이터 접근과 출력 계약을 쓰고, Core는 호출 순서를 실행합니다.

요청이 반복되고 끝나는 과정

노드와 메시지를 따라 한 번의 Run을 읽습니다.

run()이 Graph를 여는 과정

한 번의 Run마다 상태와 리소스가 따로 준비됩니다.

agent/abstract.py · agent/__init__.py · 내부 호출 경로 축약

```
await agent.run(prompt, deps=repo)
```

```
Agent.iter(...)
# _prepare_run(...)
```

```
_PreparedAgentRun.open(...)
# self.graph.iter(...)
```

```
async for node in agent_run:
    ...
```

① 앱의 진입점
run은 노드들을 끝까지 진행해
AgentRunResult를 반환.

② 실행 준비
모델, 상태, 도구와 Capability 등
이 실행에서 사용할 구성을 준비.

③ 리소스 열기
실행 범위 안에서 리소스를 열고
첫 UserPromptNode를 Graph에 전달.

④ 순회와 정리
노드가 다음 작업을 돌려줌.
실행 컨텍스트를 빠져나오며 리소스 정리.

중간 노드를 직접 보고 싶다면 async with agent.iter(...)를 씁니다.

pydantic-graph가 Agent 루프를 돌리는 방법

GraphBuilder가 구조를 만들고, 각 노드가 다음 실행을 선택합니다.

_agent_graph.py · GraphBuilder 등록과 노드 역할 축약

```
g = GraphBuilder(state_type=GraphAgentState,
                 deps_type=GraphAgentDeps, ...)
g.add(g.edge_from(g.start_node).to(UserPromptNode), ...)
```

```
UserPromptNode.run(ctx)
```

```
ModelRequestNode._prepare_request(...)
ModelRequestNode._make_request(...)
```

```
CallToolsNode._handle_tool_calls(...)
# ModelRequestNode / SetFinalResult 경로
```

① GraphBuilder
Graph 타입과 시작점을 정의.
GraphRun이 노드의 실행 결과를 받아 진행.

② UserPromptNode
이력을 불러오고 새 사용자 요청을 구성.
승인 재개라면 보류 결과도 처리.

③ ModelRequestNode
요청용 메시지·옵션을 준비하고 모델 호출.
응답을 이력에 넣어 CallToolsNode로 전달.

④ CallToolsNode
도구 반환이나 수정 요청은 다음 모델 입력.
최종 결과는 SetFinalResult에서 End로 넘김.

Graph는 실행 순서를 관리하고, Agent 노드는 모델 응답의 의미를 판단합니다.

Graph 상태와 앱 데이터가 만나는 곳

LOT-A를 조회하는 도구에는 무엇이 전달될까요?

_agent_graph.py · 런타임 객체의 역할 발췌

```
GraphAgentState
# message_history, usage, run_step
GraphAgentDeps
# model, end_strategy, usage_limits
ctx.deps.allowed_lots  # 앱 예시
ctx.deps.snapshot_id
```

실행 중 누적 상태
모델과 주고받은 이력, 사용량,
진행 단계를 현재 실행에서 누적.

실행 구성
어떤 모델과 한도, 종료 정책을
현재 Graph 실행에서 사용할지 보관.

도구가 읽는 값
도구는 앱이 넣은 권한과 데이터 기준을 읽음.
모델에는 도구가 반환한 값이 전달될 수 있음.

모델의 lot_id 제안과 앱의 조회 권한을 분리합니다.

모델 응답을 어느 경로로 보낼까?

응답의 내용과 앱의 출력 계약을 함께 확인합니다.

CallToolsNode · 실제 처리 메서드와 결과 예시

```
_handle_tool_calls(...)
# read_counts({"lot_id": "LOT-A"})
_process_response_output(...)
# text / file output path
RetryPromptPart(...)
DeferredToolRequests(...)
```

업무 도구 요청
도구 반환을 메시지에 추가.
다음 ModelRequestNode에서 모델이 읽음.

최종 출력 후보
텍스트·파일은 출력 계약에 맞춰 검사.
출력용 Tool은 Tool 호출 처리 경로에서 검사.

수정 또는 보류
수정 요청은 다음 모델 호출에 전달.
승인 대기는 앱에 넘겨 후속 실행에서 해결.

도구와 출력 후보가 함께 오면 EndStrategy가 실행 범위를 결정합니다.

다음 모델 요청이 막히는 지점

사용량 제한, 출력 재시도, 실행 취소의 효과가 다릅니다.

usage.py · _agent_graph.py · 실제 검사 이름

```
UsageLimits(request_limit=50)
check_before_request(usage)
consume_output_retry(...)
run_cancelled_snapshot(...)
```

모델 요청 횟수
기본 요청 한도 50회. 한도에 도달하면
다음 모델 요청 전에 UsageLimitExceeded.

출력 재시도
출력 검증 실패는 해당 재시도 예산을 사용.
모델 요청 횟수 한도와 별개.

실행 취소
취소 시점에 기록 가능한 실행 상태를 정리.
이미 외부에 저장한 결과의 복구는 앱 책임.

실행이 중단돼도 이미 끝난 작업은 남을 수 있습니다.

실행 API 다섯 가지

앱이 어느 시점에 무엇을 받아야 하는지에 따라 고릅니다.

PydanticAI 공개 API · Python 사용 형태

```python
result = agent.run_sync(prompt)
```

```python
result = await agent.run(prompt)
```

```python
async with agent.run_stream(prompt) as stream:
    async for part in stream.stream_output(): ...
```

```python
async for event in agent.run_stream_events(prompt): ...
```

```python
async with agent.iter(prompt) as run:
    async for node in run: ...
```

run_sync
동기 함수에서 최종 결과를 기다림.

run
비동기 서버에서 최종 결과와 이력을 받음.

run_stream
출력을 점진적으로 표시.
최종 출력 확정 시점에 유의.

run_stream_events
모델 응답과 도구 실행 이벤트를
같은 스트림에서 소비.

iter
현재 Graph 노드를 보며 실행.
run.result에서 최종 결과 확인.

출력 UI는 stream, 실행 경계 관찰은 events 또는 iter가 적합합니다.

모델 입력과 대화 상태

공급자에게 보내는 값과 앱에 저장하는 값을 나눕니다.

LOT-A 요청에서 모델이 실제로 받는 값

같은 실행을 API 요청과 Python 서버로 나눠 봅니다.

① 메시지

예: “LOT-A 수율을 알려줘”
이전 조회 결과가 있다면 이력으로 함께 전송

② 도구·출력 계약

read_counts의 lot_id는 문자열
YieldReport 필드는 출력 모드에 맞게 전송

③ Python 실행 환경

ctx.deps에 허용 LOT와 DB 연결
도구가 실행될 때 이 객체로 접근 권한 검사

④ 다음 모델 요청

도구가 반환한 good=920, total=1000
ToolReturnPart가 이력에 합쳐져 모델에 전달

모델이 읽는 값은 Python 객체가 아니라 그 실행에서 조립된 요청입니다.

새 Run에서 어떤 지시가 남을까?

첫 Run 뒤 모델 설정을 교체하는 예시로 비교합니다.

Python 호출 예시 · result.all_messages()를 다음 Run에 전달

```
a = Agent(model, instructions="단위를 표시",
          system_prompt="반도체 검사 담당자")
b = Agent(model, instructions="표로 작성")
await b.run("다시 보여줘",
            message_history=result.all_messages())
_reevaluate_dynamic_prompts(...)
```

첫 실행의 두 지시

Instructions는 현재 실행의 지시로 계산.
System prompt는 메시지 Part로 기록 가능.

다른 Agent로 계속 실행

새 instructions는 “표로 작성”.
기존 SystemPromptPart는 넘긴 이력에 남음.

동적 System Prompt

현재 Agent가 해당 동적 프롬프트를
인식하는 경로에서 다시 평가.

지시를 교체할 때는 현재 Agent 설정과 전달한 메시지 이력을 함께 확인합니다.

deps로 조회 범위를 전달하기

인증과 스냅샷은 서버에서 결정하고 Tool은 그 범위 안에서 조회합니다.

앱 코드 발췌 · 서버가 구성한 조회 범위

```
@dataclass
class Deps:
    allowed_lots: set[str]
    snapshot: str
    counts: dict[str, tuple[int, int]]

    def read_counts(self, lot_id: str):
        if lot_id not in self.allowed_lots:
            raise PermissionError(lot_id)
        good, total = self.counts[lot_id]
        return dict(good=good, total=total,
                    snapshot=self.snapshot)
```

앱에서 권한 검사

Tool 스키마에는 lot_id

인증과 스냅샷은 서버에서 결정하고 Tool은 그 범위 안에서 조회합니다.

History와 Conversation의 차이

다음 Run에 넘기는 상태의 범위를 고릅니다.

구분	실행에서 맡는 역할
message_history	이전 모델 메시지
Conversation	메시지·usage·ID·보류 호출
Agent 객체	공통 실행 설정

다음 Run에 넘기는 상태의 범위를 고릅니다.

대화 불러오기 → 실행 → 새 버전 저장

합성 LOT-A 대화를 새 프로세스에서도 이어 가는 코드입니다.

앱 저장소 예시 · ConversationTypeAdapter는 공개 API

```
old = store.read(user_id, session_id)
conversation = ConversationTypeAdapter.validate_json(old.data)
result = await agent.run("그 LOT 다시 조회",
                         conversation=conversation)
data = ConversationTypeAdapter.dump_json(result.conversation)
store.save_if_version(session_id, old.version, data)
```

① 불러오기
사용자와 세션을 확인한 뒤 저장된 JSON을
메시지·실행 상태가 있는 Conversation으로 복원.

② 이어서 실행
이전 요청·응답을 참고해 새 메시지 추가.
빈 세션은 conversation=None으로 시작.

③ 갱신
갱신한 상태를 다시 저장.
버전 확인과 충돌 처리는 앱 예시.

대화 상태는 Core에서 직렬화하고, 저장 범위와 동시 갱신은 앱이 결정합니다.

PYDANTIC AI / 04

Tool 등록과 실제 실행

함수 선언부터 인자 검증과 병렬 실행까지 추적합니다.

함수 한 개에서 스키마와 검증기가 만들어지는 과정

read_counts의 Python 선언을 모델이 읽을 수 있게 바꿉니다.

_function_schema.py · 모델 요청 필드 예시

```
def read_counts(ctx: RunContext[Repo], lot_id: str):
    """지정한 LOT의 검사 수량을 읽는다."""
function_schema(...)
{"properties": {"lot_id": {"type": "string"}},
 "required": ["lot_id"]}
```

함수 선언
함수 이름과 설명을 읽음.
lot_id는 모델이 넘길 입력.

Core 내부 준비
함수 서명과 타입을 바탕으로
JSON Schema와 입력 검증기를 준비.

모델에 공개하는 값
모델은 이름·설명·인자 계약을 받음.
함수 본문과 Repo는 Python에 남음.

같은 선언으로 모델에 공개할 계약과 호출할 때 사용할 검증기를 만듭니다.

tool과 tool_plain 선택하기

RunContext를 받는지 먼저 판단합니다.

앱 코드 발췌 · RunContext 유무 비교

```python
@agent.tool
def read_counts(ctx: RunContext[Deps], lot_id: str):
    return ctx.deps.read_counts(lot_id)

@agent.tool_plain
def yield_percent(good: int, total: int) -> float:
    return 100 * good / total
```

tool · 실행 Context 사용

tool_plain · 인자만 사용

RunContext를 받는지 먼저 판단합니다.

다음 모델 요청에서 도구 목록을 바꾸기

사용자 권한에 따라 발행 도구의 노출을 제어하는 예입니다.

tool_manager.py · toolsets/function.py · 실제 메서드

```
ToolManager.for_run_step(ctx)
FunctionToolset.get_tools(ctx)
async def prepare_publish(ctx, definition):
    return definition if ctx.deps.can_publish else None
FunctionToolset.call_tool(...)
```

요청 단계마다 목록 준비
현재 RunContext에 맞게
모델에 공개할 도구 정의를 준비.

prepare 콜백으로 숨기기
권한이 없으면 None을 반환해
그 요청의 공개 목록에서 제외.

실행할 때 다시 확인
공개 목록에 있었다고 해서 실제 작업이
지금도 허용된다는 뜻은 아님.

노출 시점에는 선택지를 줄이고, 실행 시점에는 변경된 권한을 확인합니다.

Tool 인자가 틀리면 어디에서 멈출까?

같은 read_counts 호출에 대해 실패와 성공을 비교합니다.

tool_manager.py · 호출 경계와 모델 인자 예시

```
validate_tool_call(...)
# _resolve_tool(...) / _build_tool_context(...)
_validate_tool_args(...)
validator.validate_json(...)  # 또는 validate_python
_run_validate_hooks(...)
execute_tool_call(...)
```

① 도구 찾기
도구 이름을 현재 실행에서 공개한 목록과 대조.
도구가 없으면 함수에 진입하지 않음.

② 인자 검증
{}는 필수 lot_id 누락.
검증 오류는 RetryPromptPart로 돌려보냄.

③ 정책 Hook과 실행
{"lot_id": "LOT-A"}가 검증을 통과한 뒤
서버에서 업무 권한을 확인하고 조회.

스키마 검증은 인자 형식을 확인하고, 업무 인가는 도구 또는 정책 경계에서 수행합니다.

타입 검사 이후의 업무 조건

LOT ID가 문자열이어도 조회 가능 여부는 별도 판단입니다.

구분	실행에서 맡는 역할
입력 검증	lot_id 타입 확인
업무 인가	허용 LOT·snapshot 확인
결과 검증	단위·분모·근거 확인

LOT ID가 문자열이어도 조회 가능 여부는 별도 판단입니다.

병렬 조회 사이에 단독 작업을 넣기

같은 모델 응답의 A/B 조회, C 저장, D/E 조회를 예로 봅니다.

_tool_execution.py · 실행기 로직 축약

```python
@agent.tool(sequential=True)
def save_report(ctx: RunContext[Repo], ...): ...
```

```python
_segment_by_barriers(indices, is_barrier=...)
# [A, B] → [C] → [D, E]
```

```python
for segment in segments:
    tasks = [asyncio.create_task(call_tool(i))
             for i in segment]
    # 완료/버퍼 이벤트 소비 후 다음 묶음
```

① 도구에 단독 실행 표시
C 앞의 조회가 끝나야 C를 시작.
C가 끝나기 전에는 D/E를 시작하지 않음.

② 실행 묶음 계산
실행기는 모델이 낸 호출 순서에서
단독 도구를 기준으로 묶음을 분할.

③ 독립 호출을 동시에 시작
독립 I/O는 대기 시간을 겹칠 수 있음.
공유 상태 쓰기나 호출량 폭주는 앱에서 제어.

sequential은 실행 순서를 정합니다. 앞 도구 결과를 뒷 도구 인자로 자동 전달하지 않습니다.

Local, MCP, Native Tool의 실행 위치

모델이 요청한 다음 실제 작업이 어디서 일어나는지 봅니다.

구분	실행에서 맡는 역할
Function Tool	Python 호스트 함수
MCP Tool	클라이언트가 서버 호출
Native Tool	공급자 측 기능

모델이 요청한 다음 실제 작업이 어디서 일어나는지 봅니다.

PYDANTIC AI / 05

출력 검증과 종료 정책

결과 계약과 실행할 작업을 함께 판단합니다.

출력 모드 네 가지

요청에 계약을 실어 보내는 방식부터 달라집니다.

구분	실행에서 맡는 역할
ToolOutput	출력 전용 Tool schema 공개
NativeOutput	공급자 구조화 출력 계약
PromptedOutput	지침으로 JSON 유도 후 파싱
TextOutput	문자열을 콜백으로 처리

요청에 계약을 실어 보내는 방식부터 달라집니다.

범용 Agent에서 최종 결과 타입 고르기

중간 Tool의 반환값과 앱이 받는 최종 결과를 따로 설계합니다.

구분	실행에서 맡는 역할
자유 대화	str
업무별 결과	YieldReport 또는 다른 타입
복수 형태	출력 후보를 명시

중간 Tool의 반환값과 앱이 받는 최종 결과를 따로 설계합니다.

수율과 근거를 원본에서 다시 검증하기

스키마를 통과한 90%도 원본이 92%라면 수정이 필요합니다.

앱 코드 발췌 · 수율 재계산

```python
@agent.output_validator
def verify(ctx: RunContext[Deps], out: YieldReport):
    counts = ctx.deps.read_counts(out.lot_id)
    expected = 100 * counts['good'] / counts['total']
    if out.snapshot != counts['snapshot']:
        raise ModelRetry('snapshot을 확인하세요.')
    if abs(out.yield_pct - expected) > 0.01:
        raise ModelRetry('수율을 다시 계산하세요.')
    return out
```

합성 집계 · 92 / 100

90% 보고서는 수정 요청

스키마를 통과한 90%도 원본이 92%라면 수정이 필요합니다.

같은 모델 응답에서 무엇을 실행할까?

모델 응답에 조회 A → 유효한 출력 O1 → 기록 B → 출력 O2가 왔다고 가정합니다.

_tool_execution.py · process_tool_calls · 정책별 처리 요약

```
Agent(..., end_strategy="early")
# O1 채택 / A, B, O2 생략
Agent(..., end_strategy="graceful")
# A 실행 → O1 채택 → B 실행 / O2 생략
Agent(..., end_strategy="exhaustive")
# A, O1, B, O2 모두 처리 / O1 채택
```

early
출력 도구를 먼저 검사.
유효한 출력이 있으면 업무 함수는 생략.

graceful · 기본
모델이 낸 순서에 따라 함수 묶음을 처리.
O1을 얻어도 뒤의 업무 함수는 실행 가능.

exhaustive
단독 실행 경계를 제외하면 함께 시작.
유효한 출력 선택은 응답에 나온 순서 기준.

기록·발행을 막아야 할 때는 종료 정책과 별도로 도구 자체에 승인 경계를 둡니다.

최종 출력 후보가 있어도 재요청하는 경우

출력 실패, 함수 retry, 스트림 확정은 서로 다른 분기입니다.

구분	실행에서 맡는 역할
invalid Output	다음 후보·수정 요청
function retry	정책에 따라 결과 억제
stream commit	확정 시점 예외

출력 실패, 함수 retry, 스트림 확정은 서로 다른 분기입니다.

Retry 횟수는 어느 층에서 셀까?

Tool 수정 요청과 공급자 HTTP 재시도는 각각 다른 예산을 씁니다.

구분	실행에서 맡는 역할
Core	Tool·출력 retry와 UsageLimits
전송	SDK·transport·모델 fallback
앱	업무 재시도·멱등 키

Tool 수정 요청과 공급자 HTTP 재시도는 각각 다른 예산을 씁니다.

Capability와 승인

공통 정책을 조합하고 보류된 실행을 이어갑니다.

Capability가 묶는 구성요소

반복되는 실행 정책을 여러 Agent에 연결합니다.

구분	실행에서 맡는 역할
구성	Toolset·instructions·settings
Hook	모델·Tool·출력 경계
수명	Agent 구성과 run별 바인딩

반복되는 실행 정책을 여러 Agent에 연결합니다.

Capability의 앞·뒤·감싸기 순서

정렬한 두 Capability A와 B의 모델 요청 Hook을 비교합니다.

구분	실행에서 맡는 역할
before	A → B
after	B → A
wrap	A 진입 → B 진입 → 호출 → 역순 복귀

정렬한 두 Capability A와 B의 모델 요청 Hook을 비교합니다.

업무 정책을 Hook에 넣는 위치

검증, 마스킹, 지연 측정을 필요한 경계에 둡니다.

앱 코드 발췌 · 검증된 LOT 인자 검사

```python
from pydantic_ai.capabilities import AbstractCapability

class LotPolicy(AbstractCapability[Deps]):
    async def before_tool_execute(
        self, ctx, *, call, tool_def, args
    ):
        if call.tool_name == 'read_counts':
            if args['lot_id'] not in ctx.deps.allowed_lots:
                raise PermissionError(args['lot_id'])
        return args
```

검증된 인자 검사

공통 정책으로 재사용

검증, 마스킹, 지연 측정을 필요한 경계에 둡니다.

가드레일을 어느 경계에 걸까?

요청과 도구 인자, 최종 수치는 서로 다른 시점에 확인할 수 있습니다.

Guardrail Capability · A17 보고서 예시

```
InputGuardrail.wrap_model_request(...)
ToolGuardrail.before_tool_execute(...)
OutputGuardrail.after_output_process(...)
```

입력 검사
예: 생산 데이터 반출 요청 차단.
병렬 모드는 모델 요청이 이미 시작될 수 있음.

도구 검사
예: publish 호출을 승인 대기로 전환.
allow / block / retry / approve / replace.

출력 검사
예: 최종 불량률이 원본과 맞는지 확인.
partial 출력은 이 검사에서 건너뜀.

부작용을 막는 검사는 실행 앞에 두고, 수치 검증은 최종 출력 경계에서 합니다.

승인 필요 여부를 결정하는 곳

승인이 필요한 호출은 검증된 인자와 함께 보류합니다.

구분	실행에서 맡는 역할
정적 정책	requires_approval 설정
동적 정책	ApprovalRequired로 보류
보류 결과	DeferredToolRequests 전달

승인이 필요한 호출은 검증된 인자와 함께 보류합니다.

승인 요청을 처리하는 세 경로

실행 환경에 맞는 승인 주체를 연결합니다.

구분	실행에서 맡는 역할
Inline	HandleDeferredToolCalls로 처리
CLI	사람에게 응답을 받아 전달
앱 화면	보류 저장 후 후속 Run

실행 환경에 맞는 승인 주체를 연결합니다.

승인한 호출 ID로 실행 이어가기

보고서 발행 요청을 저장하고, 앱의 승인 화면에서 결정하는 경우입니다.

Core 공개 API · 승인 결과를 전달하는 Python 예시

```python
pending = result.output  # DeferredToolRequests
call = pending.approvals[0]
```

```python
decision = DeferredToolResults(
    approvals={call.tool_call_id: True})
```

```python
await agent.run(conversation=result.conversation,
                deferred_tool_results=decision)
```

보류된 호출 저장
호출 ID와 원래 인자를 보관.
실행한 결과 대신 승인 요청이 앱에 도착.

사람이 결정
원래 호출 ID에 승인 또는 거절을 연결.
승인 권한과 유효기간은 앱에서 검사.

새 Run으로 재개
Core는 보류했던 호출과 결정을 연결.
승인 결과를 처리한 뒤 실행을 이어감.

버전이 바뀐 초안에는 이전 승인을 재사용하지 않도록 앱에서 관리합니다.

선택 Harness 기능

필요한 기억, 작업 환경, 복구 기능을 연결합니다.

Core 위에 선택 Harness 붙이기

Core 2.61.0과 Harness 0.61.0은 별도 패키지입니다.

구분	실행에서 맡는 역할
Core	Agent·Graph·Tool·Output
Harness	Memory·Skills·Guardrail
앱	필요한 기능과 운영 정책 선택

Core 2.61.0과 Harness 0.61.0은 별도 패키지입니다.

Harness Memory에 메모 한 줄 쓰기

모델이 파일 이름을 제안하고, 저장 범위는 앱이 결정합니다.

Harness memory/_capability.py · memory/_toolset.py

```
Memory(store=store, namespace="fab-seminar")
```

```
write_memory(...)
# 예: 다음 검사에서 진공 경보 확인
```

```
read_memory(...) / search_memory(...)
```

앱에서 연결
실행에서 사용할 저장소와 scope를 결정.
기본 InMemoryStore는 프로세스 수명.

모델의 쓰기 요청
Toolset이 인자를 검사하고 scope 내에서
메모 저장 또는 수정 처리.

다시 조회
정확한 파일을 읽거나 검색어로 노트를 찾음.
원본 대화 전체와는 별도의 저장 대상.

스토어를 바꾸면 메모의 수명과 공유 범위가 달라집니다.

저장한 Memory가 다음 모델 입력에 들어가는 곳

메모 파일이 모델이 읽을 수 있는 메시지로 바뀌는 경계입니다.

memory/_capability.py · 실제 메서드

```
Memory.for_run(ctx)
Memory.resolve_scope(ctx)
Memory.before_model_request(...)
Memory._load_snapshot(...)
Memory._remove_previous_injection(...)
```

실행 범위 정하기
현재 실행에서 쓸 저장소와 scope를 결정해 다른 실행의 메모와 구분.

요청 전에 읽기
메모를 읽고 줄 수와 토큰 예산 안에서 현재 요청에 붙일 문맥을 준비.

메시지로 넣기
중복 주입을 정리하고 사용자 입력 근처에 기억 문맥을 추가하는 경로.

저장된 메모 전체를 보내는 대신, 주입 범위와 예산을 설정합니다.

과거 대화 검색과 노트 검색

같은 검색어라도 조회 대상과 반환 근거가 다릅니다.

구분	실행에서 맡는 역할
대화 검색	Conversation source의 발췌
순위 계산	구간·토큰화·BM25
노트 검색	MemoryStore의 항목

같은 검색어라도 조회 대상과 반환 근거가 다릅니다.

대화 압축 전후를 메시지로 보기

오래된 검토 이력 뒤에 A17 조회가 이어진 예시입니다.

SlidingWindowCompaction · 메시지 예시는 설명용

```
[오래된 검토 1, 오래된 검토 2, A17 질문,
 read_fab_bin(A17), ToolReturn(42/1000)]
SlidingWindowCompaction.before_model_request(...)
SlidingWindowCompaction.compact(...)
[A17 질문, read_fab_bin(A17), ToolReturn(42/1000)]
```

압축 전
전체를 보내면 오래된 요청과 결과도
모델 입력에 들어가 토큰을 사용.

임계조건에서 처리
크기와 보존 조건을 검사한 뒤
남길 최근 턴을 선택.

압축 후 예
현재 질문을 뒷받침하는 최근 도구 호출과
반환을 함께 유지하는 예.

정확한 오래된 수치가 필요하면 저장소에서 다시 조회합니다.

Harness Skill이 실행에 들어오는 과정

목록에서 이름을 고르는 단계와 본문이 로딩되는 단계를 구분합니다.

skills/_capability.py · _loader.py · Skill 연결 예

```
Skills.for_run(...)
# SKILL.md → 이름·설명
load_capability(...)
# 예: fab-bin-review
SKILL.md → references/...
```

목록 준비
실행에 연결된 Skill을 발견하고
메타데이터로 사용할 절차를 안내.

필요한 것 선택
모델이 절차를 선택하면 해당 Capability의
본문이나 연결 기능을 활성화.

참고파일 접근
세부 절차가 필요할 때 파일 접근.
경로와 실행 권한은 연결한 환경에 따름.

발견된 목록, 실제로 로딩된 본문, 도구 실행 권한을 구분합니다.

파일·Shell·계획 기능 조합하기

업무에 필요한 작업 환경만 Agent에 연결합니다.

구분	실행에서 맡는 역할
Workspace / FileSystem	파일 범위와 접근
Shell / Sandbox	명령 실행 환경과 제한
Planning	작업 계획을 구조화

업무에 필요한 작업 환경만 Agent에 연결합니다.

서버가 멈췄을 때 무엇부터 복구할까?

같은 발행 작업에서 복구 단위를 비교합니다.

Conversation

사용자 요청·도구 호출·결과와 보류 상태
앱이 이 상태를 다음 Run에 전달

StepPersistence

완료한 단계와 효과를 checkpoint에 기록
복구 시 어디까지 했는지 결정하는 데 사용

Durable 실행

실행기가 정한 경계에서 replay 또는 재개
공급자 호출과 도구 효과를 구분해 설정

앱의 중복 방지

보고서 ID·버전으로 멱등 키 지정
발행 성공 후 응답을 잃어도 중복 발행 차단

대화가 저장됐다는 사실만으로 외부 작업의 중복 실행을 막을 수는 없습니다.

PYDANTIC AI / 08

위임, 관측, 개발 범위

자식 실행과 운영 경계를 같은 사례에 연결합니다.

위임과 Handoff에서 다음 작업을 정하는 곳

자식 결과의 행선지와 제어권을 비교합니다.

구분	실행에서 맡는 역할
Agent-as-tool	부모에게 결과 반환
앱 Handoff	앱이 다음 Agent 선택
업무 Graph	명시적 전이 규칙

자식 결과의 행선지와 제어권을 비교합니다.

위임 요청을 자식 Run으로 실행하기

예를 들어 통계 검토자에게 A17의 42/1000을 넘깁니다.

subagents/_toolset.py · 실행 경계와 업무 예시

```
delegate_task(...)
# “A17 42/1000의 불확실성 검토”
SubAgentToolset._run_delegation(...)
ToolReturnPart(...)
# “42/1000 = 4.2%”
```

부모에서 업무 구성
자식에게 필요한 데이터와 목표를 명시.
부모의 전체 이력을 공유한다고 가정하지 않음.

자식 실행
선택된 자식의 Agent 실행.
Deps 전달과 사용량 추적은 설정을 확인.

부모에 결과 반환
부모가 도구 결과를 읽고 답변을 종합.
위임과 별도 앱 Handoff의 차이.

부모는 결과를 받은 뒤 다음 결정을 내립니다.

Dynamic Workflow와 Advisor 선택하기

작업 그래프를 진행하는 경우와 조언을 받는 경우를 구분합니다.

구분	실행에서 맡는 역할
Dynamic Workflow	단계와 의존성으로 작업 진행
Advisor	별도 관점에서 검토 의견
부모 Agent	결과 취합과 다음 판단

작업 그래프를 진행하는 경우와 조언을 받는 경우를 구분합니다.

Trace로 모델과 Tool 실행 이어 보기

Run에서 어떤 호출과 검증이 발생했는지 확인합니다.

구분	실행에서 맡는 역할
Run span	요청 전체의 상관관계
Model / Tool span	호출 지연·사용량·오류
Output / 검증	처리와 오류 경계

Run에서 어떤 호출과 검증이 발생했는지 확인합니다.

흐름 테스트와 답변 평가

고정 응답으로 실행을 재현하고 데이터셋으로 답변을 평가합니다.

구분	실행에서 맡는 역할
TestModel	기본 연결 검사
FunctionModel	원하는 응답·오류 재현
Evals	사례별 수치·근거·품질 평가

고정 응답으로 실행을 재현하고 데이터셋으로 답변을 평가합니다.

어디까지 개발해야 Agent가 될까?

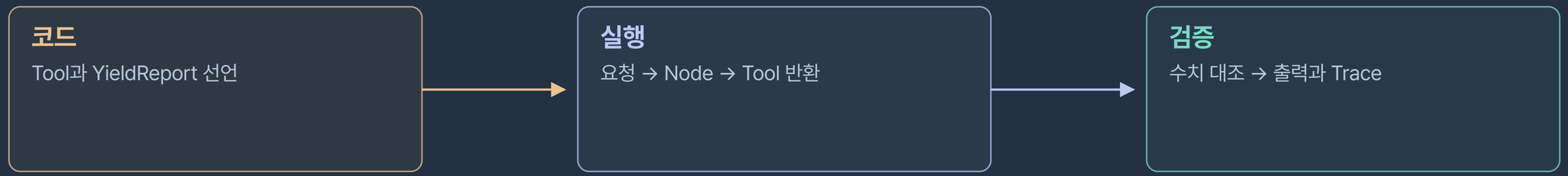
최소 실행에서 운영 서비스까지 책임이 늘어나는 지점을 봅니다.

구분	실행에서 맡는 역할
최소 Agent	모델·조회 Tool·출력 타입
업무 Agent	인가·수치 검증·대화·승인
서비스	저장 충돌·복구·관측 운영

최소 실행에서 운영 서비스까지 책임이 늘어나는 지점을 봅니다.

코드와 내부 실행을 한 요청으로 따라가기

합성 LOT-A를 조회하고 수율 보고서를 검증합니다.



합성 LOT-A를 조회하고 수율 보고서를 검증합니다.

SDK 실행 경계 비교

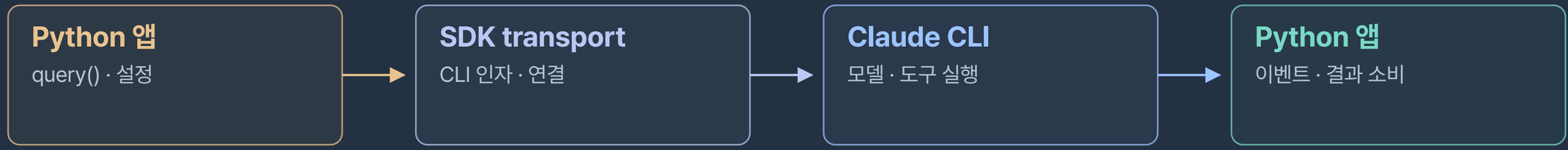
PydanticAI의 LOT-A 조회를 네 SDK에서 이어갑니다.

실행 지점	관찰할 동작	확인할 조건
실행 주체	Claude CLI · ADK Runner	호스트와 Agent 런타임의 경계
업무 제어	ADK Graph · Microsoft Workflow	이벤트 반영과 단계 복구
한 턴의 분기	OpenAI Runner	도구 결과 · 종료 · 활성 Agent

공통 입력은 LOT-A, snapshot r3, 양품 92 / 전체 100입니다.

Claude Agent SDK의 프로세스 경계

Claude Code 실행환경을 Python 업무 앱에 연결합니다.



실행 지점	관찰할 동작	확인할 조건
Python 앱	query()와 설정·콜백 정의	SDK가 CLI 명령 구성
Claude CLI	작업 폴더에서 모델·도구 실행	CLI 내부 루프는 공개 계약으로 설명
메시지와 제어	stdout 메시지는 앱이 소비	권한·SDK MCP 요청은 Python으로 전달

CLI 버전과 작업 환경까지 배포 계약에 포함됩니다.

단발 요청과 대화형 클라이언트

같은 LOT 조회에서 입력과 연결 수명을 비교합니다.

Python API 발췌 · import와 업무 정의 생략

```python
async for message in query(prompt=prompt, options=options):
    if isinstance(message, ResultMessage):
        print(message.subtype)

async with ClaudeSDKClient(options=options) as client:
    await client.query(prompt)
    async for message in client.receive_response():
        if isinstance(message, ResultMessage):
            print(message.subtype)
```

query()

비동기 메시지 반복자

한 번의 실행 흐름을 소비

ClaudeSDKClient

connect → query → receive_response

연결을 유지하며 후속 입력 전달

결과 처리

ResultMessage와 오류 확인

연결 수명은 앱에서 관리

두 API 모두 결과 메시지를 끝까지 소비합니다.

Python 조회 함수를 SDK MCP로 연결

LOT-A 조회의 실제 실행 위치를 따라갑니다.

Python API 발췌 · import와 업무 정의 생략

```python
@tool("read_lot", "Read an allowed LOT", {"lot_id": str})
async def read_lot(args):
    report = read_allowed_lot(args["lot_id"])
    return {"content": [{"type": "text", "text": report}]}

server = create_sdk_mcp_server(name="lot", tools=[read_lot])
options = ClaudeAgentOptions(
    mcp_servers={"lot": server},
    allowed_tools=["mcp__lot__read_lot"],
)
```

Python 등록

@tool + create_sdk_mcp_server

업무 함수는 호스트 프로세스에 존재

Claude CLI

mcp__lot__read_lot 호출 요청

SDK의 control 채널로 전달

Python 디스패치

_handle_sdk_mcp_request

함수 결과를 CLI로 반환

PydanticAI의 직접 함수 등록과 호출 경계가 다릅니다.

작업 환경과 도구 권한

CLI 정책과 LOT 데이터 인가를 구분합니다.

실행 지점	관찰할 동작	확인할 조건
작업 환경	cwd와 setting_sources 지정	프로젝트 설정 로딩 범위 확인
호출 정책	도구 목록·permission mode	Hook과 can_use_tool 개입점 확인
데이터 인가	read_lot 내부에서 LOT 범위 검사	CLI 허용 이후에도 검증

권한 콜백에서 거절한 호출은 업무 함수 실행 0회를 기대합니다.

세션 재개와 산출물 버전

세션을 이어도 LOT 보고서의 근거는 별도로 확인합니다.

실행 지점	관찰할 동작	확인할 조건
세션	session_id로 대화 재개	fork는 대화 분기
작업 환경	cwd와 실제 파일 상태 확인	실행 파일과 산출물을 따로 식별
업무 근거	LOT-A · snapshot r3 · 92/100	보고서 revision과 저장 위치 기록

대화 이력과 업무 근거의 식별자를 함께 기록합니다.

스트림과 구조화 결과

앱은 메시지 스트림을 읽고 업무 결과를 검증합니다.

실행 지점	관찰할 동작	확인할 조건
StreamEvent	부분 메시지는 기본 비활성	include_partial_messages=False
ResultMessage	종료 subtype과 structured_output	실행 결과와 JSON 결과 구분
앱 검증	허용 LOT·snapshot·수율 재확인	검증 후 저장 또는 실패 기록

구조화된 JSON에도 LOT와 원본 버전 검증이 필요합니다.

Runner와 이벤트 스트림

한 invocation에서 응답과 상태 변경 이벤트를 함께 읽습니다.

Python API 발췌 · import와 업무 정의 생략

```python
runner = Runner(agent=agent, app_name="yield_demo",
                session_service=session_service)
await session_service.create_session(
    app_name="yield_demo", user_id="operator", session_id="r3")
async for event in runner.run_async(
    user_id="operator", session_id="r3", new_message=message
):
    print(event.author, event.actions.state_delta)
```

호스트

Runner.run_async() 호출

user_id와 session_id 전달

Agent 실행

BaseLlmFlow가 모델·함수 처리

이벤트로 진행을 전달

세션 서비스

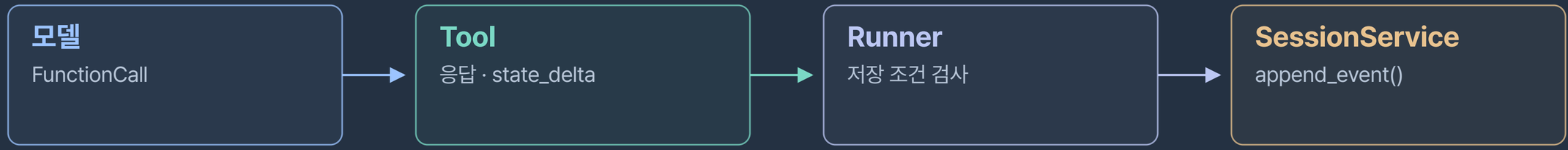
Runner가 저장 후보 이벤트 처리

클라이언트에도 이벤트 전달

이벤트는 진행 관찰과 상태 기록에 모두 쓰입니다.

함수 응답과 상태 반영

Tool 응답이 모델과 세션에 전달되는 경로를 추적합니다.



실행 지점	관찰할 동작	확인할 조건
모델	FunctionCall에 도구 이름·인자	LLM flow에서 실행으로 연결
Tool	FunctionResponse와 state_delta	응답 내용과 변경값을 분리
Runner → SessionService	부분 이벤트 등을 선별	append_event에서 상태 반영

> 부분 스트림과 저장된 이벤트는 서로 다른 목록입니다.

Session·State·Memory·Artifact

하나의 분석에서 저장 대상을 나눕니다.

실행 지점	관찰할 동작	확인할 조건
Session과 State	대화 이벤트와 업무 값	app: / user: / 세션 / temp: 구분
Memory	이전 정보의 검색 경로	세션 이력과 별도 서비스
Artifact	보고서 파일과 버전	상태에는 참조 식별자 기록

상태 스코프와 실제 저장 서비스가 데이터 수명을 결정합니다.

Agent 전이와 고정 순서

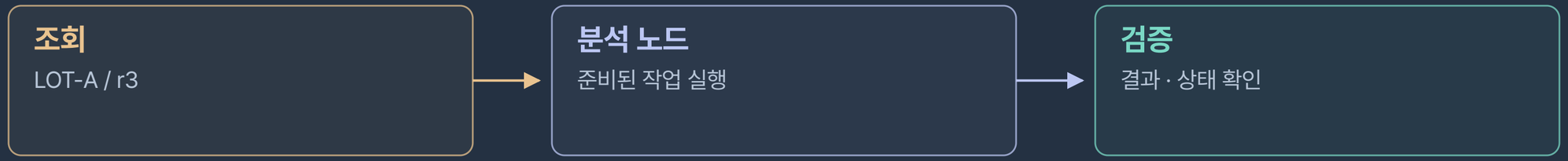
다음 단계를 고르는 위치에 따라 실행을 선택합니다.

실행 지점	관찰할 동작	확인할 조건
AgentTool	부모 Agent가 결과를 받아 계속 실행	도구로 하위 Agent 호출
Agent transfer	모델이 전이 요청	대상 Agent로 실행 흐름 연결
Workflow Agent	Sequential / Parallel / Loop	코드에서 단계 배치

모델에 맡기는 선택과 업무 순서를 코드에서 정하는 선택을 구분합니다.

ADK 2.x Graph 스케줄러

업무 그래프에서 준비된 노드가 실행되는 지점을 읽습니다.



실행 지점	관찰할 동작	확인할 조건
Node와 Edge	조회 → 병렬 분석 → 검증	입력 의존성과 시작 조건 선언
_schedule_ready_nodes()	준비 조건과 동시 실행 한도 확인	선택된 노드 작업 시작
_emit_node_checkpoint()	노드 진행 상태 기록 경로	재실행·복구 조건은 별도 확인

PydanticAI 내부 루프 Graph와 ADK 업무 Graph는 역할이 다릅니다.

답변 완료와 발행 완료

보고서 저장에 실패하는 조건으로 업무 종료를 확인합니다.

실행 지점	관찰할 동작	확인할 조건
응답 이벤트	모델이 수율 92%를 설명	is_final_response() 판정
저장 시도	Artifact 저장 실패 주입	발행 완료 상태 기록 보류
검수 기록	실제 남은 이벤트와 Artifact 조회	발행 횟수 0회 기대

최종 응답과 업무 완료를 서로 다른 조건으로 검사합니다.

실행 설정과 이벤트 검증

LLM 호출 한도와 로컬 재현 조건을 함께 기록합니다.

Python API 발췌 · import와 업무 정의 생략

```python
config = RunConfig(max_llm_calls=3)
async for event in runner.run_async(
    user_id="operator",
    session_id="r3",
    new_message=message,
    run_config=config,
):
    if event.is_final_response():
        print(event.author)
```

RunConfig

streaming_mode=NONE

max_llm_calls 기본 500

환경·실행 경로

ADK_MAX_LLM_CALLS로 기본값 변경

Live 호출 한도는 별도 검토

이벤트 시험

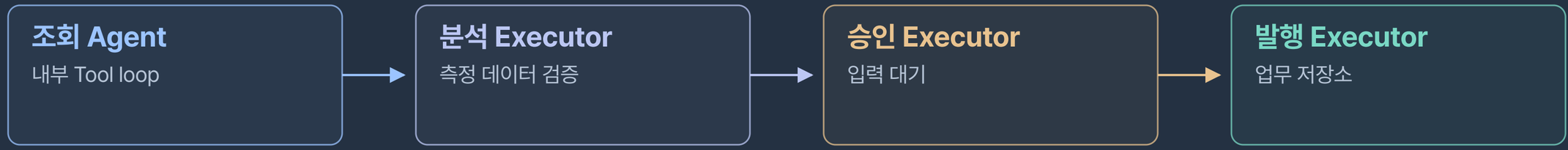
모델 응답 고정 · 실제 Runner 사용

state_delta와 산출물 참조 검증

테스트는 고정 모델 응답과 실제 서비스 연동을 구분합니다.

Agent 실행과 업무 Workflow

조회 Agent 한 번의 run을 업무 그래프에 넣습니다.



실행 지점	관찰할 동작	확인할 조건
Workflow	조회 → 분석 → 승인 → 발행	Executor와 Edge로 단계 제어
AgentExecutor	Agent를 한 Executor로 사용	내부 모델·도구 루프 실행
상태 경계	AgentSession은 대화 상태	Workflow는 메시지·복구 위치 관리

업무 순서와 복구 조건을 그래프에 드러낼 수 있습니다.

Executor와 Edge의 메시지 계약

업무 단계 사이에서 값의 타입과 의미를 검사합니다.

Python API 발췌 · import와 업무 정의 생략

```
builder = WorkflowBuilder(start_executor=read_lot)
builder.add_edge(read_lot, analyze)
builder.add_edge(analyze, approve)
builder.add_edge(approve, publish)
workflow = builder.build()

async for event in workflow.run(request, stream=True):
    print(event.type)
```

보내는 단계

Executor가 LOT 결과를 보냄

LOT-A · r3 · 측정 요약

Edge

다음 Executor로 라우팅

필요한 경우 분기 조건 적용

받는 단계

입력 타입 및 업무 계약 확인

snapshot 불일치 시 중단

타입이 맞아도 LOT와 snapshot이 일치하는지 확인합니다.

AgentExecutor의 입력과 출력

워크플로 메시지가 Agent.run 인자로 바뀌는 경로입니다.

실행 지점	관찰할 동작	확인할 조건
from_message()	Message를 핸들러가 받음	Agent 실행 입력으로 전달
_run_agent_and_emit()	Agent 응답 및 사용자 입력 요구 처리	대화 Session과 실행 연결
후속 Executor	AgentExecutorResponse 전달 경로	출력 이벤트를 업무 데이터와 구분

AgentExecutor는 워크플로와 Agent 실행 계약을 연결합니다.

병렬 결과를 결합하는 업무 키

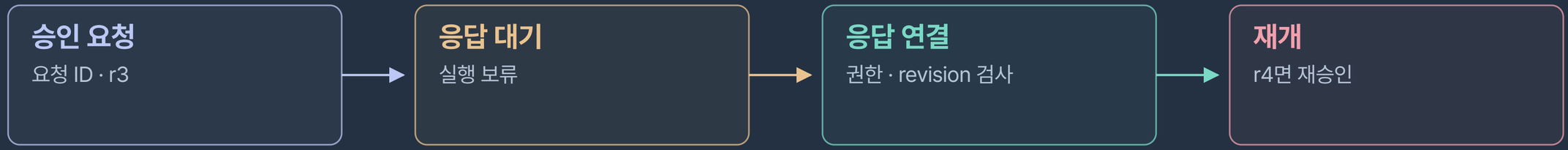
온도·압력 분석의 도착 순서를 바꿔 검증합니다.

실행 지점	관찰할 동작	확인할 조건
Fan-out	같은 LOT-A / r3 전달	온도와 압력 Executor에 분배
Fan-in	업무 키로 결과를 결합	LOT + snapshot + 분석 종류
검증	지연·중복·r4 혼입 시도	r3 결과만 보고서에 사용

도착 순서와 업무상 결합 기준을 분리합니다.

승인 대기와 응답 연결

승인 요청 ID와 보고서 revision을 묶습니다.



실행 지점	관찰할 동작	확인할 조건
요청	Workflow가 사용자 입력 대기	요청 ID와 LOT-A / r3 기록
응답	같은 요청 ID로 응답을 전달	승인 대상·권한 재확인
재개	r4를 발행하려는 경우 보류	새 보고서 버전으로 재승인

r3 승인으로 r4 발행이 0회인지 확인합니다.

체크포인트와 외부 발행 복구

어디에서 복구해도 발행이 한 번인지 검사합니다.

Python API 발췌 · import와 업무 정의 생략

```python
storage = FileCheckpointStorage("./checkpoints")
builder = WorkflowBuilder(
    start_executor=read_lot, checkpoint_storage=storage
)
builder.add_edge(read_lot, analyze)
workflow = builder.build()
await workflow.run(request)
checkpoints = await storage.list_checkpoints(
    workflow_name=workflow.name
)
```

Entry checkpoint

초기 입력·응답 전달 시점

Python 1.13.0부터 추가

Superstep 완료

Executor 상태·다음 메시지 등

저장소 지정 시 복구 가능

외부 발행

멱등 키로 재실행 검증

업무 저장소에서 중복 확인

업무 발행의 중복 방지는 외부 저장소까지 포함합니다.

Agent와 명시적 Executor의 배치

업무에서 판단을 위임할 곳과 검증을 고정할 곳을 나눕니다.

실행 지점	관찰할 동작	확인할 조건
Agent	분석 근거 요약	불확실한 서술과 설명 생성
Executor	수율 재계산 · 승인 범위 검사	고정 업무 규칙 실행
저장·발행	보고서 revision과 키 기록	복구와 중복 처리 시험

명시적인 그래프에는 메시지와 저장소 관리 비용이 따릅니다.

Runner가 한 턴을 해석하는 방식

PydanticAI의 내부 Graph와 비교해 다음 상태를 읽습니다.



실행 지점	관찰할 동작	확인할 조건
모델 호출	활성 Agent와 입력으로 요청	Runner가 응답 분류
도구·handoff	도구 실행 또는 활성 Agent 교체	다음 입력과 상태 구성
종료	최종 출력 또는 한도·중단	기본 max_turns=10

한 턴은 모델 호출과 그 턴의 도구 처리를 포함합니다.

도구 결과로 종료하는 정책

같은 조회 응답으로 후속 모델 호출 유무를 비교합니다.

Python API 발췌 · import와 업무 정의 생략

```python
agent = Agent(
    name="Yield analyst",
    tools=[read_lot],
    tool_use_behavior="run_llm_again",
)
normal = await Runner.run(agent, prompt)
direct = await Runner.run(
    agent.clone(tool_use_behavior="stop_on_first_tool"), prompt
)
```

run_llm_again

기본값 · Tool 결과를 모델에 전달

모델이 후속 출력 작성

stop_on_first_tool

함수 도구 결과를 최종값으로 선택

실행된 도구 목록은 별도 기록

StopAtTools / callable

특정 이름 또는 결과 조건 사용

종료 판단 위치 확인

최종값 선택과 실행된 Tool 수를 각각 측정합니다.

Agent.as_tool과 handoff

전문가의 답변을 받는 경우와 실행 주체를 바꾸는 경우입니다.

Python API 발췌 · import와 업무 정의 생략

```
manager = Agent(
    name="Manager", tools=[expert.as_tool(
        tool_name="ask_expert", tool_description="Summarize LOT"
    )]
)
triage = Agent(
    name="Triage", handoffs=[handoff(expert, input_filter=filter_history)]
)
result = await Runner.run(triage, prompt)
print(result.last_agent.name)
```

Agent.as_tool()

하위 Agent가 실행되고 결과 반환

부모가 계속 응답

handoff()

Runner가 활성 Agent 변경

이후 Agent가 답변 책임을 가짐

입력 이력

handoff input_filter 적용 가능

모델 입력과 저장 이력 구분

누가 최종 응답을 생성하는지 추적으로 확인합니다.

Session과 RunState의 저장 경계

이전 대화와 멈춘 실행을 이어받는 입력을 구분합니다.

실행 지점	관찰할 동작	확인할 조건
Session	클라이언트가 관리하는 대화 이력	다음 Run의 입력 구성
RunState	중단된 Run의 진행 상태	승인 결정 후 Runner에 전달
Responses API 식별자	conversation_id / previous_response_id	공급자 쪽 대화 연결

> 다음 대화와 중단된 실행은 서로 다른 저장 계약을 가집니다.

Guardrail과 Tool 승인

검사 지점과 실행 대기 지점을 분리합니다.

Python API 발췌 · import와 업무 정의 생략

```python
@function_tool(needs_approval=True)
async def publish_report(lot: str, revision: str) -> str:
    return await publish_once(lot, revision)

result = await Runner.run(agent, prompt)
if result.interruptions:
    state = result.to_state()
    for item in result.interruptions:
        if approved_for_r3(item):
            state.approve(item)
    result = await Runner.run(agent, state)
```

Agent 검사

첫 Agent 입력 · 최종 Agent 출력

병렬 입력 검사는 호출 시작 가능

Function Tool 검사

도구 입력·출력 경계

handoff 경로와 구분

Approval

보류 후 RunState 승인·재개

외부 발행 전 업무 범위 확인

승인 전후의 실제 Tool 실행 횟수를 확인합니다.

Trace에서 실행 주체 확인

동일 LOT 요청의 모델·도구·전이 기록을 연결합니다.

실행 지점	관찰할 동작	확인할 조건
Trace	한 업무 실행의 관찰 단위	Agent 및 generation span
전이와 도구	handoff / function span	전이 전후 활성 Agent 확인
수집 설정	RunConfig에서 tracing 제어	민감 데이터 포함 여부 명시

Trace 설정과 업무 감사 기록의 저장 정책을 분리합니다.

고정 모델 응답으로 실행 비교

SDK 규칙과 자연 모델의 선택을 분리해 검증합니다.

실행 지점	관찰할 동작	확인할 조건
Tool 종료 비교	같은 응답으로 정책만 변경	모델 횟수·도구 횟수 비교
Agent 협업 비교	부모 반환과 handoff 비교	최종 Agent·전달 이력 확인
중단과 재개	승인 전후 RunState 추적	발행 중복 여부 별도 검증

작은 API로 조합할 수 있고, 이력과 저장소 선택이 실행 계약을 바꿉니다.

Agent를 어디에서 실행할까?

중앙 서버와 사용자 PC에 배치했을 때의 차이

중앙 Agent 실행

공통 운영 + 사용자·작업 격리

Client → API / Queue

인증·tenant·세션 순서

Agent Workers

Tool / DB · 작업 수명·예산

사용자 로컬 실행

설치·업데이트 + 로컬 실행 권한

Desktop Agent

서명된 배포·업데이트·정책 버전

로컬 Tools / 저장소

사용자 파일·실행 계정 권한

모델 서비스 · 부하·접근 제어

Agent 실행 자원과 모델 서버 자원을 나누어 살펴봅니다.

Application

실행 기록과 업무 결과 검증

무엇을 실행했는지, 요구한 결과가 맞는지 확인합니다.

Trace · 무엇이 실행되었나

관찰 가능한 실행 증거

실행 기록

Run / Request / Call ID
실제 인자·Tool 결과
승인·재개·오류 경로
사용한 데이터 snapshot

Evaluation · 옳게 수행했나

업무 목표와 제약 기준

업무 검증

분자·분모·기간 일치
근거·SOP revision 확인
승인 인자와 실행 일치
예측 시간 분할·누수 확인

같은 Tool도 데이터와 기준 시점이 다르면 다른 결과를 냅니다.

데이터 범위, 계산, 근거, 승인 내용을 함께 확인합니다.

구조를 이해하고 실제 개발로

Hermes와 PydanticAI에서 살펴본 내용

01 **Hermes**

상태·기억·도구·정책의 동작

02 **PydanticAI**

코드 정의부터 실행과 검증까지

03 **업무 적용**

데이터·권한·결과 검증

3차에서는 Agent를 개발하고 검증 결과와 시행착오를 공유합니다.